PORTFOLIO

SLMAN AL-ODANI

NHL STENDEN EMMEN
Computer Science
Student Number : 5638364
Number of EC's : 240
Slman.al-odani@student.nhlstenden.com
2025-2029

Introduction:

As a Computer Science student, this portfolio reflects the journey I've taken to explore, build, and understand the digital world. From foundational programming concepts to hands-on projects, each section showcases not just what I've learned, but how I've applied it. Whether it's solving problems through code, analyzing systems, or experimenting with new technologies, I've aimed to approach each challenge with curiosity and persistence. This collection is a snapshot of my growth and a starting point for where I hope to go next.

My GitHub name is slmanalodani

Table of Contents

Introduction: 1

Computer science assignments: 5

- Setup guides: 5
  - PlatformIO setup guide for Arduino projects: 5
  - Git Setup Guide for Sharing Projects: 6
- Week 0: Scratch: 8
- Week 1: C 12
  - Assignment1: Hello, it's me 12
  - Assignment2: Mario-less 12
  - Assignment3: Cash 15
- Week 2: Arrays 18
  - Assignment1: Scrabble 18
  - Assignment2: Readability 20
  - Assignment3: Caesar 23
- Week 3: Algorithms 25
  - Assignment1: Sort 25
  - Assignment2: Plurality 26
  - Assignment3: Runoff 29
- Week 4: Memory 35
  - Assignment1: Filter less 35
  - Assignment2: Recover 39
- Week 5: Data structures 42
  - Assignment1: Git Together 42
  - Assignment2: Inheritance 43
  - Assignment3: speller 47

Embedded assignments: 52

CuteBot Race game: 52
Mario uno: 54
Morse code : 56
Advance traffic lights: 60
Displaying Uptime: 64
(Realtime) Jumbo Clock: 70
Weather station: 75
Professional Skills: 80

Computer science assignments:

Setup guides:

PlatformIO setup guide for Arduino projects:

1. Install VS Code:
   Download and install Visual Studio Code: https://code.visualstudio.com/

2. Install PlatformIO Extension:
   Open VS Code -> Go to extensions -> search for PlatformIO IDE ->
   Click Install.

3. Create a New Project:
   Click the PlatformIO icon in the sidebar -> select "New Project" -> Name your project (e.g, BlinkLED) -> choose your board (e.g, Arduino Uno R4 wifi) -> Select the framework: Arduino -> click "Finish".

4. Edit main.cpp:
   Inside the src folder, write your code:
   This is example for blinking code:

```
#include <Arduino.h>

// put function declarations here:
int myFunction(int, int);

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}

// put function definitions here:
```

```
int myFunction(int x, int y) {
  return x + y;
}
```

5. Connect Your Arduino:
   Use a USB cable (USB-C or A depending on your board).

6. Upload the Code:
   Click the "Upload" arrow in the top right corner.

Git Setup Guide for Sharing Projects:

1. Install Git

Download and install Git from: https://git-scm.com/downloads

2. Initialize a Git Repository

Open your project folder in your terminal and run:

```
git init
```

This creates a. git folder and starts tracking your project.

3. Add Your Files

Stage all files for commit:

```
git add .
```

4. Commit Your Work

Save your changes with a message:

```
git commit -m "Initial commit"
```

5. Create a GitHub Repository

Go to https://github.com → click the **+ icon** → choose **New repository** Name it (e.g. GitTestProject) → choose **Public** or **Private** → click **Create repository**

6. Link Your Local Repo to GitHub

Copy the URL GitHub gives you (e.g., https://github.com/yourname/GitTestProject.git) Then run:

```
git remote add origin https://github.com/yourname/GitTestProject.git
```

7. Push Your Code

Send your code to GitHub:

```
git push -u origin master
```

8. Share Your Repository

Copy and share the GitHub URL — anyone can view or clone it.

Week 0: Scratch:

The assignment is to make a game using scratch following these guidelines:

1. Your project must use at least two sprites.
2. Your project must have at least three scripts total (i.e., not necessarily three per sprite).
3. Your project must use at least one conditional, at least one loop, and at least one variable.
4. Your project must use at least one custom block that you have made yourself (via **Make a Block**), which must take at least one input.
5. Your project should be more complex than most of those demonstrated in the lecture.

Day	Logbook of work
One	I watched the lecture, and I decided to make a move and collect items while dodging monsters.
Two	I chose the sprites and the background.
Three	I started coding.
Four	I completed the coding and submitted the game

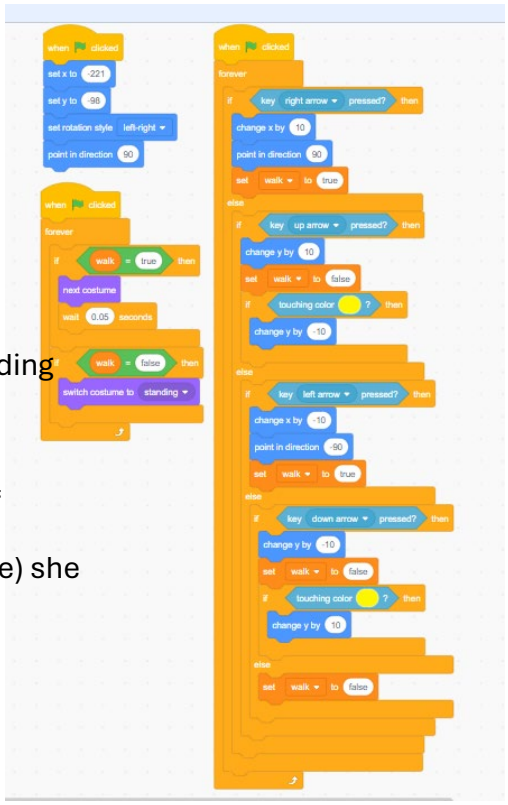
- The codes for my controlled sprite (Avery):

  → I started with the x and y coordinates codes to make her spawn at a certain point.

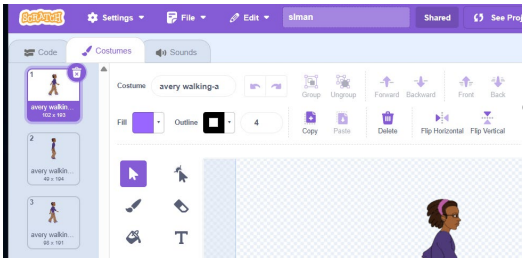
  → I used the point in a direction code to make her face the right direction by setting the value to 90.

  → I used the rotational style (left – right) code so she doesn't spin or rotate in a circle. Instead, she flips horizontally depending on the direction.

  → I gave her the walking animation through a combination of forever loop, if condition when I want it her to walk (walk = true) she changes to the next costume.



Also, I set 0.05 seconds wait and it is fast enough to



feel responsive, but slow enough that each costume is

visible. She has 5 costumes and last one is for the

standing animation, so I set an if condition when

(walk = false) she shows it.

→ I want her to move when the arrow keys are pressed

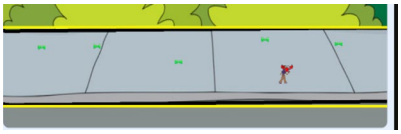
so, I used the following code combinations:

- Forever loop for continues movement.
- If condition when a key arrow is pressed combined with the change of x or y axes by

(10 or –10). She moves to the right by 10 units and she faces the right direction through the code (point in direction 90) and added to that the animation through (set the walk variable to true) code.

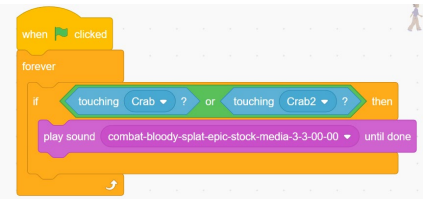
I did the same for the left direction movement just by changing the (10 and 90) to (–10 and –90).

For the up and down movements, I want her to not go past the two yellow lines, so I used the if condition touching the yellow color code it changes the y value by the opposite amount that make her go up or down.

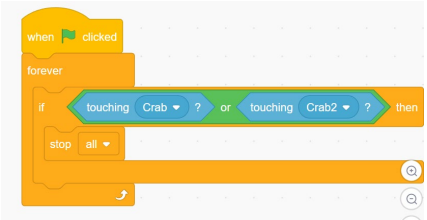


→ This is the code that I used to play a sound when she touches

two certain sprites and in this case crab and crab2.



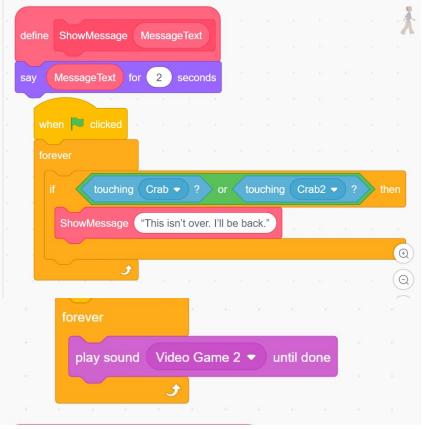
→ The purpose of this code is to make the game end when Avery touches crab or crab2.



→ I used this code to make her show a message when she touches crab or crab2 so basically when the game ends.

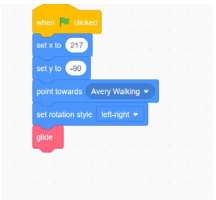
- Other codes for the Background.

→ This code is to play music in the background in a loop.



- The codes for the crab sprites:

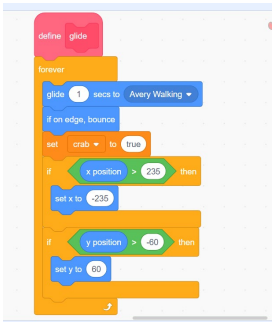
→ This code is for the red crab sprite it points toward Avery and Spawn in a certain coordinate with right and left style of rotation.



- This is the glide code:

  It allows the crab to glide towards Avery and bounce if it hits an edge.

Also, it prevents it from crossing the yellow lines and contains animation.



→ This is the animation code the red crab.

→ This is the text code for the red crab.

→ These two codes are for the blue crab:

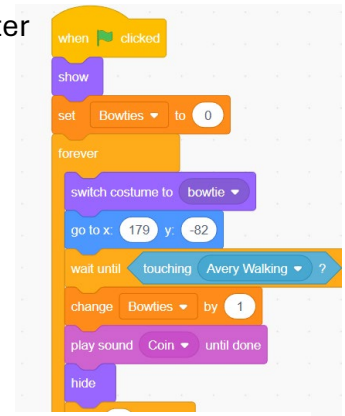
After 20 sec from starting the game, it spawns and glide towards Avery while talking to the other crab.

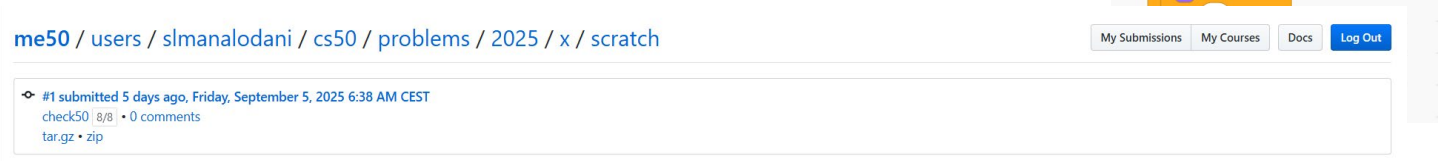


Slman Al-Odani

- This is the code for the bowties:
  I have 5 bowties they have this same code with just different spawn coordinates.

→ The idea here is to make them respawn after collecting them with a counter in the top left of the screen, and when Avery touches them that will cause a sound.





https://submit.cs50.io/check50/7e9355c48419cdedc2708af67df093c6722a985a

https://scratch.mit.edu/projects/1212465656

Self-reflection (week 0.1 – scratch):

- This week I made a simple game on Scratch, and I learned how to use loops, conditionals, and events. It was tricky at first, but I figured things out by testing and fixing my mistakes. I'm starting to enjoy coding and I'm curious to see what's next.

Week 1: C

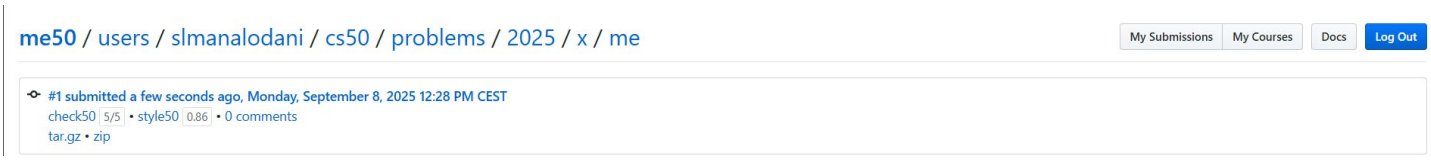
Assignment1: Hello, it's me

In a file called `hello.c`, in a folder called `me`, implement a program in C that prompts the user for their name and then says hello to that user.

The code:

```c
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    string firstname = get_string("What is your name?\n");
    printf("hello, %s\n", firstname);          // to print hello + string
}
```



https://submit.cs50.io/check50/fbdcc5782c01b82cc6f12cd5519b85d69d992624

Assignment2: Mario-less

- Toward the end of World 1-1 in Nintendo's Super Mario Bros., Mario must ascend a right-aligned pyramid of bricks.
- In a file called `mario.c` in a folder called `mario-less`, implement a program in C that recreates that pyramid, using hashes (#) for bricks.
- But prompt the user for an int for the pyramid's actual height, so that the program can also output shorter pyramids.
- Re-prompt the user, again and again as needed, if their input is not greater than 0 or not an int altogether.

Day	Logbook of work
One	I watched the lecture
Two	I started coding
Three	I completed the coding.

The code:

```c
#include <cs50.h>
#include <stdio.h>

void print_row(int spaces, int bricks); // Custom function to make spaces and bricks
int main(void)

{

    int n; // to define n as integer

    do // do while loop to keep asking the user for the intended answer for the height.

    {
        n = get_int("Height: ");
    }
    while (n < 1 || n > 8); // to always get positive number and not more than 8

    for (int i = 0; i < n; i++)

    {

        int numberspaces =
            n - 1 - i; // formula to calculate the number of spaces to print before the
bricks

        int numberbricks = i + 1; // formula to calculate the number of bricks

        print_row(numberspaces, numberbricks); // to print a row with a certain number of
spaces
                                               // followed by a certain number of bricks
    }
}
```

```c
void print_row(int spaces, int bricks)

{

    // print spaces

    for (int i = 0; i < spaces; i++)

    {

        printf(" ");
    }

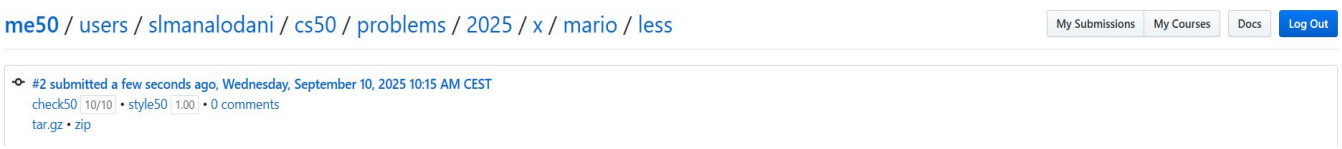
    // print bricks

    for (int i = 0; i < bricks; i++)

    {

        printf("#");
    }

    printf("\n");
}
```



https://submit.cs50.io/check50/be84e87a6625066cee4d9edae7ebe0b025e8d19c

Assignment3: Cash

- In a file called cash.c in a folder called cash, implement a program in C that prints the minimum coins needed to make the given amount of change, in cents,
- Prompt the user for an int greater than 0, so that the program works for any amount of change.
- Re-prompt the user, again and again as needed, if their input is not greater than or equal to 0 (or if their input isn't an int at all!).

The code:

```c
#include <cs50.h>
#include <stdio.h>

int calculate_quarters(int cents); //  declare custom function to calculate the quarters.
int calculate_dimes(int cents);
int calculate_nickels(int cents);
int calculate_pennies(int cents);
int main(void)
{

    int cents;
    do
    {
        cents = get_int("changed owed: "); // Prompt the user for change owed in cents
    }
    while (cents < 1); // to get only positive numbers and greater than zero

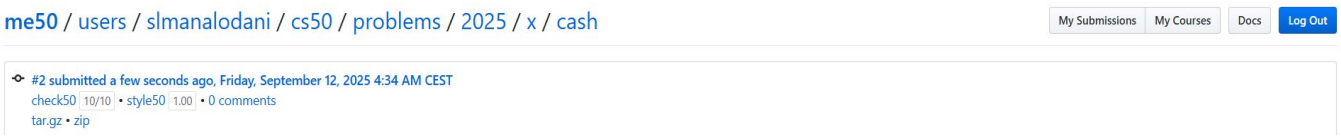
    int quarters =
        calculate_quarters(cents);   // Calculate how many quarters you should give customer
    cents = cents - (quarters * 25); // Subtract the value of those quarters from cents
    int dimes = calculate_dimes(cents);
    cents = cents - (dimes * 10);
    int nickels = calculate_nickels(cents);
    cents = cents - (nickels * 5);
    int pennies = calculate_pennies(cents);
    cents = cents - (pennies * 1);

    int totalNumberCoins = quarters + dimes + nickels +
                           pennies;   // Sum the number of quarters,dimes,nickels and pennies
used
    printf("%i\n", totalNumberCoins); // print the sum
```

```c
}

int calculate_quarters(int cents)

{
    int quarters = 0;   // define quarters as integer and we start from 0.
    while (cents >= 25) // loop to add 1 quarter if we have 25 cents.
    {
        quarters++;
        cents = cents - 25;
    }
    return quarters; // to return quarter value when called
}
int calculate_dimes(int cents)
{
    int dimes = 0;
    while (cents >= 10)
    {
        dimes++;
        cents = cents - 10;
    }
    return dimes;
}
int calculate_nickels(int cents)
{
    int nickels = 0;
    while (cents >= 5)
    {
        nickels++;
        cents = cents - 5;
    }
    return nickels;
}
int calculate_pennies(int cents)
{
    int pennies = 0;
    while (cents >= 1)
    {
        pennies++;
        cents = cents - 1;
    }
    return pennies;
}
```



https://submit.cs50.io/check50/8529f9507f111c9332226d65bd75ae2af39ca002

Week 2: Arrays

Assignment1: Scrabble

In a file called scrabble.c in a folder called scrabble, implement a program in C that determines the winner of a short Scrabble-like game. Your program should prompt for input twice: once for “Player 1” to input their word and once for “Player 2” to input their word. Then, depending on which player scores the most points, your program should either print “Player 1 wins!”, “Player 2 wins!”, or “Tie!” (in the event the two players score equal points).

Code:

```
#include <cs50.h>
#include <ctype.h> // for toupper()
#include <stdio.h>
#include <string.h> // for strlen()

// points assigned to each letter of the alphabet
int POINTS[] = {1, 3, 3, 2, 1, 4, 2, 4, 1, 8, 5, 1, 3, 1, 1, 3, 10, 1, 1, 1, 1, 4, 4, 8, 4,
10};

int compute_score(string word);

int main(void)
{
    // prompt the user for two words
    string word1 = get_string("player 1: ");
    string word2 = get_string("player 2: ");

    // compute the score of each word
    int score1 = compute_score(word1);
    int score2 = compute_score(word2);

    // print the winner
    if (score1 > score2)
    {
        printf("player 1 wins!\n");
    }
    else if (score1 < score2)
    {
        printf("player 2 wins!\n");
    }

    else
```

```
    {
        printf("Tie!\n");
    }

    return 0; // to tell the computer we are done
}

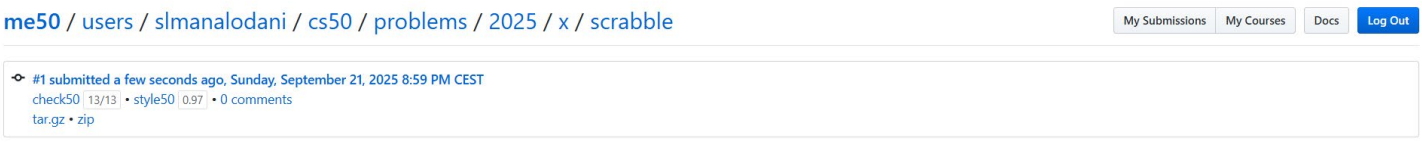
int compute_score(string word)
{
    // keep track of the total
    int total = 0;

    int length = strlen(word); // get the number of characters the word has without the null
terminator

    for (int i = 0; i < length; i++) // loop to go through each character
    {
        char c = toupper(word[i]); // convert character uppercase and store it in c

        if (c >= 'A' && c <= 'Z') // check if the character is between the letter (A-Z)
        {
            int position = c - 'A'; // turn the letter into its alphapet position (A = 0)

            total += POINTS[position]; // calculate the total points of word
        }
    }
    return total; // return the final value
}
```



https://submit.cs50.io/check50/97683d3f22319669fa5f0a5125e0de65655bd52e

Assignment2: Readability

In a file called readability.c in a folder called readability, you'll implement a program that calculates the approximate grade level needed to comprehend some text. Your program should print as output "Grade X" where "X" is the grade level computed, rounded to the nearest integer. If the grade level is 16 or higher (equivalent to or greater than a senior undergraduate reading level), your program should output "Grade 16+" instead of giving the exact index number. If the grade level is less than 1, your program should output "Before Grade 1".

Code:

```c
#include <cs50.h>
#include <ctype.h>
#include <math.h> // for round()
#include <stdio.h>
#include <string.h> // for strlen()

int count_letters(string text);
int count_words(string text);
int count_sentences(string text);

int main(void)
{
    // prompt the user for some text
    string text = get_string("Text: \n");

    // count the number of letters, words, and sentences in the text
    int letters = count_letters(text);
    int words = count_words(text);
    int sentences = count_sentences(text);

    // compute the Coleman-Liau index
    float L = (letters / (float) words) * 100;
    float S = (sentences / (float) words) * 100;
    float index = 0.0588 * L - 0.296 * S - 15.8;

    // print the grade level
    int grade = round(index);
    if (grade < 1)
    {
        printf("Before Grade 1\n");
    }
```

```
    else if (grade >= 16)
    {
        printf("Grade 16+\n");
    }
    else
    {
        printf("Grade %i\n", grade);
    }
}
int count_letters(string text)
{

    int letters = 0; // keep track of the letters

    int length = strlen(text); // return the length of the text and store it in length

    for (int i = 0; i < length; i++)
    {

        if (isalpha(text[i])) // check if the text has letters in the index postition
        {
            letters++; // 1+
        }
    }

    return letters;
}

int count_words(string text)
{

    int words = 1; // one because start to count the first word

    int length = strlen(text);

    for (int i = 0; i < length; i++)
    {
        if (text[i] == ' ') // check if the text has a space in the index postition
        {
            words++;
        }
    }
```

```
    return words;
}

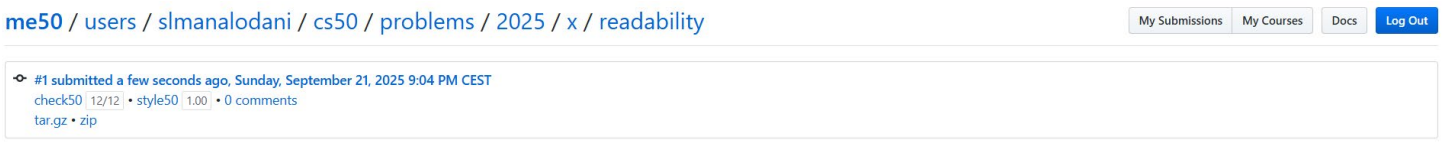
int count_sentences(string text)
{

    int sentences = 0; // keep track of the sentences

    int length = strlen(text);

    for (int i = 0; i < length; i++)
    {
        if (text[i] == '?' || text[i] == '.' || text[i] == '!') // sentences end usually with
these.
        {
            sentences++;
        }
    }

    return sentences;
}
```



https://submit.cs50.io/check50/8be259a234b396d5bb897e3afd0fe29bc486362a

Assignment3: Caesar

In a file called caesar.c in a folder called caesar, write a program that enables you to encrypt messages using Caesar's cipher. At the time the user executes the program, they should decide, by providing a command-line argument, what the key should be in the secret message they'll provide at runtime. We shouldn't necessarily assume that the user's key is going to be a number; though you may assume that, if it is a number, it will be a positive integer.

Code:

```
#include <cs50.h>  // For isupper
#include <ctype.h> // For isdigit and isalpha
#include <stdio.h>
#include <stdlib.h> // For atoi
#include <string.h>

int main(int argc, string argv[]) /*argc tells how many command line arguments were passed
argv[0] the name of the program, and argv[1] is the key*/

{
    if (argc != 2) // To accept only 2 command line arguments. (! = not)
    {
        printf("Usage: ./caesar key \n");
        return 1;
    }

    int length = strlen(argv[1]);
    for (int i = 0; i < length; i++) // Loop through every character
    {
        if (!isdigit(argv[1][i])) // Check if key is not integer
        {
            printf("Usage: ./caesar key \n");
            return 1;
        }
    }
    int key = atoi(
        argv[1]); // Convert a string of digits into integer to be able to use them in the
formula

    string plaintext = get_string("plaintext:  "); // Prompt the user for plaintext

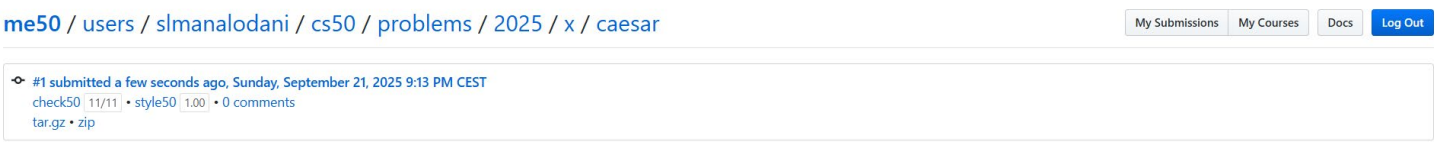
    printf("ciphertext: ");

    for (int i = 0; i < strlen(plaintext); i++) // Go through each character in the plaintext
```

```
    {
        char c = plaintext[i]; // To store current character in c to work with it easily

        if (isalpha(c)) // To check if the character is a letter eaither uppercase or
lowercase
        {
            char base =
                isupper(c) ? 'A' : 'a'; /* Setting the base ASCII value
    if c is uppercase base = 65 else base = 97 - ( ? and : )ternary statement shorter way to
if else condition*/

            printf("%c", (c - base + key) % 26 +
                           base); /* (c - base): letter index - %26 It keeps the result
inside the
range 0-25, which maps to letters 'A' to 'Z'. Encrypt and print the shifted letter*/
        }
        else
        {
            printf("%c", c); // print anything that is not letters
        }
    }
    printf("\n");
}
```



https://submit.cs50.io/check50/ef6ceb06b119b7ade7a1d41803e5a311883d3903

Week 3: Algorithms

Assignment1: Sort

In this Assignment, you'll analyze three (compiled!) sorting programs to determine which algorithms they use. In a file called `answers.txt` in a folder called `sort`, record your answers, along with an explanation for each program, by filling in the blanks marked `TODO`.

My answers:

sort1 uses: Bubble sort

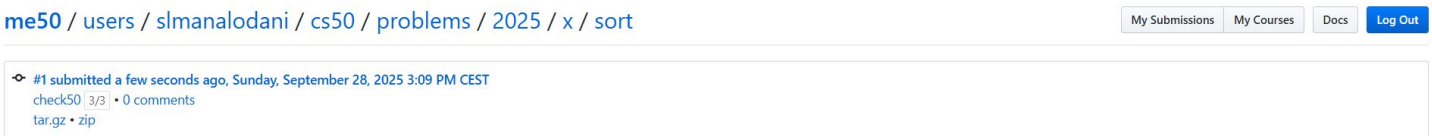
How do you know?: Bubble sort performs better when numbers are already sorted in order. I tested the sorted50000.txt and it took 0.315s while sort 3 took 2.633s.

sort2 uses: Merge sort

How do you know?: It was the fastest when it comes to random numbers which is what merge sort good at. I tested it with 50000random.txt, and it took 0.264s while the other two took much more time than that.

sort3 uses: Selection sort

How do you know?: when I tested sort3 with sorted50000.txt it took it 2.633s which is much higher than sort 1. This means this sorting is taking so much time with sorted elements and it is one of the indication that this is a selection sort.



https://submit.cs50.io/check50/844f427f698875a64da4ec8e5ba77bf41783ede3

Assignment2: Plurality

You should not modify anything else in plurality.c other than the implementations of the vote and print_winner functions.

The code:

```
#include <cs50.h>
#include <stdio.h>
#include <string.h>

// Max number of candidates
#define MAX 9

// Candidates have name and vote count
typedef struct
{
    string name;
    int votes;
} candidate;

// Array of candidates
candidate candidates[MAX];

// Number of candidates
int candidate_count;

// Function prototypes
bool vote(string name);
void print_winner(void);

int main(int argc, string argv[])
{
    // Check for invalid usage
    if (argc < 2)
    {
        printf("Usage: plurality [candidate ...]\n");
        return 1;
    }

    // Populate array of candidates
    candidate_count = argc - 1;
    if (candidate_count > MAX)
    {
```

```
        printf("Maximum number of candidates is %i\n", MAX);
        return 2;
    }
    for (int i = 0; i < candidate_count; i++)
    {
        candidates[i].name = argv[i + 1];
        candidates[i].votes = 0;
    }

    int voter_count = get_int("Number of voters: ");

    // Loop over all voters
    for (int i = 0; i < voter_count; i++)
    {
        string name = get_string("Vote: ");

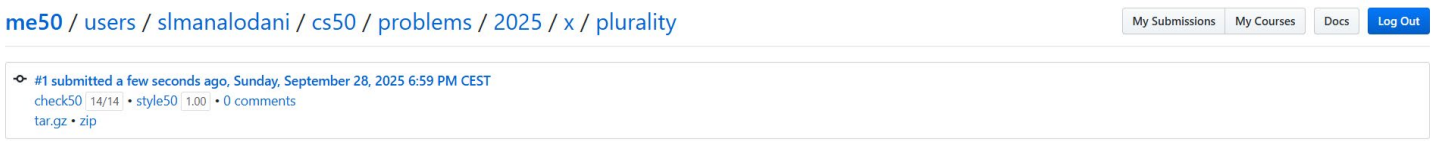
        // Check for invalid vote
        if (!vote(name))
        {
            printf("Invalid vote.\n");
        }
    }

    // Display winner of election
    print_winner();
}

// Update vote totals given a new vote
bool vote(string name)
{
    // Iterate over each candidate
    for (int i = 0; i < candidate_count; i++)
    {
        // Check if candidate's name matches given name
        if (strcmp(name, candidates[i].name) == 0)
        {
            // If yes, increment candidate's votes and return true
            candidates[i].votes++;
            return true;
        }
    }
    return false;
}
```

```c
// Print the winner (or winners) of the election
void print_winner(void)
{
    int max_votes = 0;

    // Find the maximum number of votes
    for (int i = 0; i < candidate_count; i++)
    {
        if (candidates[i].votes > max_votes)
        {
            max_votes = candidates[i].votes;
        }
    }
    // Print the candidate (or candidates) with maximum votes
    for (int j = 0; j < candidate_count; j++)
    {
        if (max_votes == candidates[j].votes)
        {
            printf("%s\n", candidates[j].name);
        }
    }
    return;
}
```



https://submit.cs50.io/check50/5acaf1ca3aa336fd9c974ba46675b2a74714b5c8

Assignment3: Runoff

In a file called runoff.c in a folder called runoff, create a program to simulate a runoff election.

The code:

```c
#include <cs50.h>
#include <stdio.h>
#include <string.h>

// Max voters and candidates
#define MAX_VOTERS 100
#define MAX_CANDIDATES 9

// preferences[i][j] is jth preference for voter i
int preferences[MAX_VOTERS][MAX_CANDIDATES];

// Candidates have name, vote count, eliminated status
typedef struct
{
    string name;
    int votes;
    bool eliminated;
} candidate;

// Array of candidates
candidate candidates[MAX_CANDIDATES];

// Numbers of voters and candidates
int voter_count;
int candidate_count;

// Function prototypes
bool vote(int voter, int rank, string name);
void tabulate(void);
bool print_winner(void);
int find_min(void);
bool is_tie(int min);
void eliminate(int min);

int main(int argc, string argv[])
{
    // Check for invalid usage
```

```
    if (argc < 2)
    {
        printf("Usage: runoff [candidate ...]\n");
        return 1;
    }

    // Populate array of candidates
    candidate_count = argc - 1;
    if (candidate_count > MAX_CANDIDATES)
    {
        printf("Maximum number of candidates is %i\n", MAX_CANDIDATES);
        return 2;
    }
    for (int i = 0; i < candidate_count; i++)
    {
        candidates[i].name = argv[i + 1];
        candidates[i].votes = 0;
        candidates[i].eliminated = false;
    }

    voter_count = get_int("Number of voters: ");
    if (voter_count > MAX_VOTERS)
    {
        printf("Maximum number of voters is %i\n", MAX_VOTERS);
        return 3;
    }

    // Keep querying for votes
    for (int i = 0; i < voter_count; i++)
    {

        // Query for each rank
        for (int j = 0; j < candidate_count; j++)
        {
            string name = get_string("Rank %i: ", j + 1);

            // Record vote, unless it's invalid
            if (!vote(i, j, name))
            {
                printf("Invalid vote.\n");
                return 4;
            }
        }
```

```
        printf("\n");
    }

    // Keep holding runoffs until winner exists
    while (true)
    {
        // Calculate votes given remaining candidates
        tabulate();

        // Check if election has been won
        bool won = print_winner();
        if (won)
        {
            break;
        }

        // Eliminate last-place candidates
        int min = find_min();
        bool tie = is_tie(min);

        // If tie, everyone wins
        if (tie)
        {
            for (int i = 0; i < candidate_count; i++)
            {
                if (!candidates[i].eliminated)
                {
                    printf("%s\n", candidates[i].name);
                }
            }
            break;
        }

        // Eliminate anyone with minimum number of votes
        eliminate(min);

        // Reset vote counts back to zero
        for (int i = 0; i < candidate_count; i++)
        {
            candidates[i].votes = 0;
        }
    }
    return 0;
}
```

```
bool vote(int voter, int rank, string name)
{
    for (int i = 0; i < candidate_count; i++)
    {
        // check if name matches a candidate
        if (strcmp(name, candidates[i].name) == 0)
        {
            preferences[voter][rank] =
                i; // the voter ranked the candidate at position i as thier rankth choice.
            return true;
        }
    }
    return false;
}

// Tabulate votes for non-eliminated candidates
void tabulate(void)
{
    for (int i = 0; i < voter_count; i++) // go through each voter
    {
        for (int j = 0; j < candidate_count; j++) // go through thier ranked choice
candidates
        {
            // get candidate index from preferences
            int candidates_index = preferences[i][j];

            // check if candidate is still in race
            if (!candidates[candidates_index].eliminated)
            {
                candidates[candidates_index].votes++;
                break; // because each voter gives one vote per round
            }
        }
    }
    return;
}

// Print the winner of the election, if there is one
bool print_winner(void)
{
    // calculate majority needed to win
    int majority = voter_count / 2 + 1;
```

```
    for (int i = 0; i < candidate_count; i++)
    {
        // check if candidate has majority
        if (candidates[i].votes >= majority)
        {
            // print winner name
            printf("%s\n", candidates[i].name);
            return true;
        }
    }
    // no winner
    return false;
}

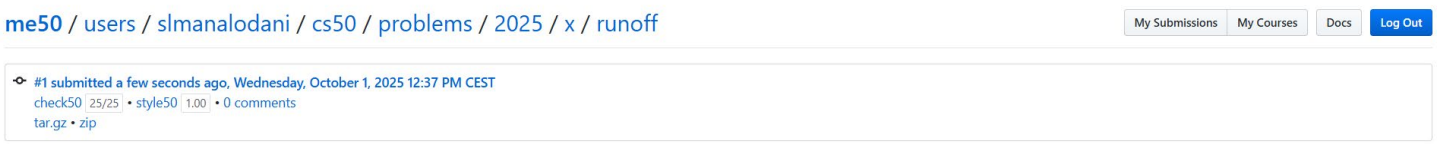
// Return the minimum number of votes any remaining candidate has
int find_min(void)
{
    // no one can have more votes than the voter_count and then we gonna loop and update the
minimum
    int min = voter_count;

    for (int i = 0; i < candidate_count; i++)
    {
        // find lowest vote among remaining candidates
        if (!candidates[i].eliminated && candidates[i].votes < min)
        {
            min = candidates[i].votes;
        }
    }
    return min;
}

// Return true if the election is tied between all candidates, false otherwise
bool is_tie(int min)
{
    for (int i = 0; i < candidate_count; i++)
    {
        // if the candidates have more votes than the min then its not a tie
        if (!candidates[i].eliminated && candidates[i].votes != min)
        {
            return false;
        }
    }
    // all remaining candidates have same votes
```

```
    return true;
}

// Eliminate the candidate (or candidates) in last place
void eliminate(int min)
{
    for (int i = 0; i < candidate_count; i++)
    {
        // eliminate candidates with min votes
        if (candidates[i].votes == min)
        {
            candidates[i].eliminated = true;
        }
    }
    return;
}
```



https://submit.cs50.io/check50/8a0e7a370f00506a0a4a16c5e7799b94a44b6035

Week 4: Memory

Assignment1: Filter less

In a file called `helpers.c` in a folder called `filter-less`, write a program to apply filters to BMPs.

The code:

```c
#include "helpers.h"
#include <math.h>

// Convert image to grayscale
void grayscale(int height, int width, RGBTRIPLE image[height][width])
{
    // calculate the avergae value
    float avg;

    // loop over all the pixels
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            avg = round((image[i][j].rgbtRed + image[i][j].rgbtGreen + image[i][j].rgbtBlue)
/
                        3.0); // image[i][j] to access any pixel in the image

            // update the pixel values making them equal to each other to get the greyscale
effect.
            image[i][j].rgbtRed = avg;
            image[i][j].rgbtGreen = avg;
            image[i][j].rgbtBlue = avg;
        }
    }
    return;
}

// Convert image to sepia
void sepia(int height, int width, RGBTRIPLE image[height][width])
{
    int sepiaRed;
    int sepiaGreen;
    int sepiaBlue;

    // loop over all the pixels
    for (int i = 0; i < height; i++)
```

```
    {
        for (int j = 0; j < width; j++)
        {

            // calculate each new color value using the sepia formula
            sepiaRed = round(.393 * image[i][j].rgbtRed + .769 * image[i][j].rgbtGreen +
                             .189 * image[i][j].rgbtBlue);
            sepiaGreen = round(.349 * image[i][j].rgbtRed + .686 * image[i][j].rgbtGreen +
                               .168 * image[i][j].rgbtBlue);
            sepiaBlue = round(.272 * image[i][j].rgbtRed + .534 * image[i][j].rgbtGreen +
                              .131 * image[i][j].rgbtBlue);

            // ensure the result is an integer between 0 and 255
            image[i][j].rgbtRed = sepiaRed > 255 ? 255 : sepiaRed;
            image[i][j].rgbtGreen = sepiaGreen > 255 ? 255 : sepiaGreen;
            image[i][j].rgbtBlue = sepiaBlue > 255 ? 255 : sepiaBlue;
        }
    }
    return;
}
// Reflect image horizontally
void reflect(int height, int width, RGBTRIPLE image[height][width])
{
    // loop over all the pixels
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width / 2; j++) // we divide by 2 to make the loop go half way so
we
                                            // swap each pixel with its mirror side.
        {
            // swap pixels
            int mirrors =
                width - j -
                1; // will give me the index of the pixel in the opposite side of the row
from j

            RGBTRIPLE tmp = image[i][j]; // Temporarily store the full pixel (RGBTRIPLE
struct)
                                         // including red, green, and blue values
            image[i][j] = image[i][mirrors];
            image[i][mirrors] = tmp;
        }
    }
    return;
```

```c
}

void blur(int height, int width, RGBTRIPLE image[height][width])
{
    // Make a copy of the original image so we don't mess up pixel values while calculating
    RGBTRIPLE copy[height][width];

    // Copy each pixel from the original image into the copy
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            copy[i][j] = image[i][j]; // we are using the copy to read the nighbor values
        }
    }

    // Now go through each pixel in the image
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            // These will hold the total color values from neighboring pixels
            int totalRed = 0;
            int totalGreen = 0;
            int totalBlue = 0;

            // This keeps track of how many pixels we added (important for edge cases) and we
use it
            // for the avg calculation
            int count = 0;

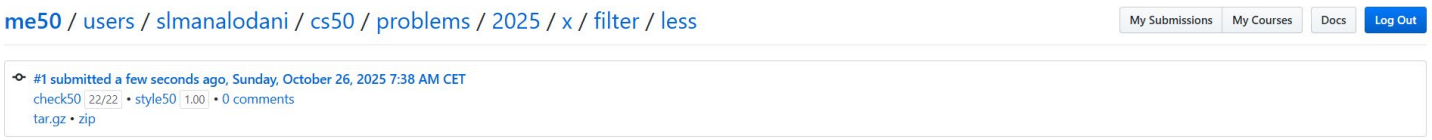
            // Loop through the 3×3 box around the current pixel
            for (int row = i - 1; row <= i + 1; row++) // (<=) to include the pixel itself
            {
                for (int col = j - 1; col <= j + 1; col++) // (<=) same here
                {
                    // Make sure the neighbor is inside the image boundaries

                    if (row >= 0 && row < height && col >= 0 && col < width) /*
                    row >= 0    prevent Negative row index (above image)
                    row < height    prevent Row index too large (below image)
                    col >= 0    prevent Negative column index (left of image)
                    col < width  prevent Column index too large (right of image) */
                    {
```

```
                        // Add up the red, green, and blue values
                        totalRed += copy[row][col].rgbtRed;
                        totalGreen += copy[row][col].rgbtGreen;
                        totalBlue += copy[row][col].rgbtBlue;

                        // Count this pixel
                        count++;
                    }
                }
            }

            // Now calculate the average and update the original image
            image[i][j].rgbtRed = round((float) totalRed / count);
            image[i][j].rgbtGreen = round((float) totalGreen / count);
            image[i][j].rgbtBlue = round((float) totalBlue / count);
        }
    }
}
```



https://submit.cs50.io/check50/01e61bfcd900d7692d6a6c19362ea5b023c6775a

Assignment2: Recover

In a file called `recover.c` in a folder called `recover`, write a program to recover JPEGs from a memory card.

The code:

```c
#include <stdint.h> // for uint8_t
#include <stdio.h>  // for fopen, fread, fwrite, sprintf
#include <stdlib.h>

#define BLOCKS 512

typedef uint8_t BYTE; // creates a nickname (alias) for an existing type.

int main(int argc, char *argv[])
{
    // only accept one command-line argument
    if (argc != 2)
    {
        printf("Usage: ./recover FILE\n");
        return 1;
    }

    // open the memory card
    FILE *card = fopen(argv[1], "r");

    // if the file failed to open, user get error
    if (card == NULL) // NULL is a speical pointer means nothing or invalid address and
doesnt point
                      // anywhare
    {
        printf("file failed to open!\n");
        return 1;
    }

    BYTE buffer[BLOCKS]; // bc card.raw store data in blocks of 512 bytes
    FILE *img = NULL; // Declare a pointer to a FILE and initialize it to NULL — i.e., it's
not
                      // pointing to any file yet.
    int file_count = 0; // counter for naming JPEG files
    char filename[8];   // to store filenames like "000.jpg" (7 + the null terminator)
```

```
    // while there's still data left to read from the memory card
    while (fread(buffer, sizeof(BYTE), BLOCKS, card) == BLOCKS) // loop condition
    {
        // Check if this block is the start of a JPEG file
        if (buffer[0] == 0xff && // first byte is 0xff
            buffer[1] == 0xd8 && // second byte is 0xd8
            buffer[2] == 0xff && // third byte is 0xff
            (buffer[3] & 0xf0) ==
                0xe0) /* fourth byte starts with 0xe here we igonre the last 4 bits
                      & : Bitwise AND operator. here it Compares each bit of buffer[3] with
0xf0.
                      and it returns a new nummber where : - Each bit is 1 only if both bits
were 1.
                      Otherwise, it's 0. ex: buffer[3] = 11101010 and 0xf0 = 11110000 so the
result
                      is 11100000 which is in this case 0xe0*/
        {
            // If we're already writing a JPEG, close it before starting a new one
            // for the first loop this condition is not met thats why we are not closing img
            // however second loop img will met the condtion and close the previous img file.
            if (img != NULL)
            {
                fclose(img);
            }
            // Create a new filename like "000.jpg", "001.jpg", etc.
            sprintf(filename, "%03i.jpg", file_count);
            // sprintf formats a string and stores it in filename.
            //%03i means: Print the integer with 3 digits, padded with zeros.

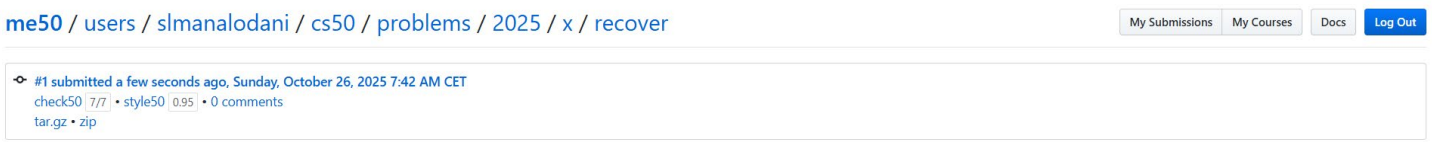
            // Open a new file to write the JPEG data
            // Here img is pointing to newly opened file
            img = fopen(filename, "w");

            // Increase the file counter for the next JPEG
            file_count++;
        }

        // If we're currently writing to a JPEG file, write the block
        if (img != NULL)
        {
            fwrite(buffer, sizeof(BYTE), BLOCKS, img); /*
             fwrite parameters:
             buffer → source of data
```

```
            sizeof(BYTE) → size of each element (1 byte)
            BLOCKS → number of elements (512)
            img → destination file */
        }
    }
    // Close any remaining open JPEG file
    if (img != NULL)
    {
        fclose(img);
    }

    // Close the memory card file
    fclose(card);
    return 0;
}
```



https://submit.cs50.io/check50/a3a717ebc5467f9c03db7b92b9ceb814ce25bb1a

Week 5: Data structures

Assignment1: Git Together

The assignment is for 4 members. One member creates a new Git-repository, and the others clone it and create a new branch at the end, merges the branches and makes the code work.

I was responsible for making a function that asks the user for 2 numbers and multiply them.

The code:

```
#include <Arduino.h>

void setup() {
  Serial.begin(9600);  // Start serial communication
}

void loop() {
  // Menu logic will call multiplyTwoNumbers() when needed
}

// Function to ask for two numbers and multiply them
void multiplyTwoNumbers() {
  int num1 = 0;
  int num2 = 0;

  Serial.println("Enter first number:");
  while (Serial.available() == 0) {}
  num1 = Serial.parseInt();

  Serial.println("Enter second number:");
  while (Serial.available() == 0) {}
  num2 = Serial.parseInt();

  int result = num1 * num2;

  Serial.print("Result: ");
  Serial.println(result);
}
```

The repository link:

https://github.com/EviTheStudent/Git-Together.git

Assignment2: Inheritance

In a file called `inheritance.c` in a folder called `inheritance`, simulate the inheritance of blood types for each member of a family.

The code:

```c
// Simulate genetic inheritance of blood type
#define _DEFAULT_SOURCE
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Each person has two parents and two alleles
typedef struct person
{
    struct person *parents[2]; //An array of two pointers to other person structs.
                               //Each pointer represents one biological parent.
    char alleles[2];
} person;

const int GENERATIONS = 3;
const int INDENT_LENGTH = 4;

person *create_family(int generations);
void print_family(person *p, int generation);
void free_family(person *p);
char random_allele();

int main(void)
{
    // Seed random number generator
    srandom(time(0));

    // Create a new family with three generations
    person *p = create_family(GENERATIONS);

    // Print family tree of blood types
    print_family(p, 0);

    // Free memory
```

```
    free_family(p);
}

// Create a new individual with `generations`
person *create_family(int generations)
{
    // Allocate memory for new person
    person *new_person = malloc(sizeof(person));

    // If there are still generations left to create
    if (generations > 1)
    {
        // Create two new parents for current person by recursively calling create_family
        person *parent0 = create_family(generations - 1); // Decreases the generation count
by 1.
        person *parent1 = create_family(generations - 1);

        // Set parent pointers for current person
        new_person->parents[0] = parent0; // Connects the first parent to the current person.
        new_person->parents[1] = parent1; // Connects the second parent to the current
person.

        // Randomly assign current person's alleles based on the alleles of their parents
        new_person->alleles[0] = parent0->alleles[random() % 2]; //gives either 0 or 1,
picking one of the parent's two alleles.
        new_person->alleles[1] = parent1->alleles[random() % 2];
    }

    // If there are no generations left to create
    else
    {
        // Set parent pointers to NULL
        new_person->parents[0] = NULL;
        new_person->parents[1] = NULL;
        // Randomly assign alleles
        new_person->alleles[0] = random_allele();
        new_person->alleles[1] = random_allele();
    }

    //  Return newly created person
    return new_person;
}

// Free `p` and all ancestors of `p`.
```

```c
void free_family(person *p)
{
    // Handle base case
    if (p == NULL)
    {
        return;
    }
    // Free parents recursively
    free_family(p->parents[0]);
    free_family(p->parents[1]);
    // Free child
    free(p);
}

// Print each family member and their alleles.
void print_family(person *p, int generation)
{
    // Handle base case
    if (p == NULL)
    {
        return;
    }

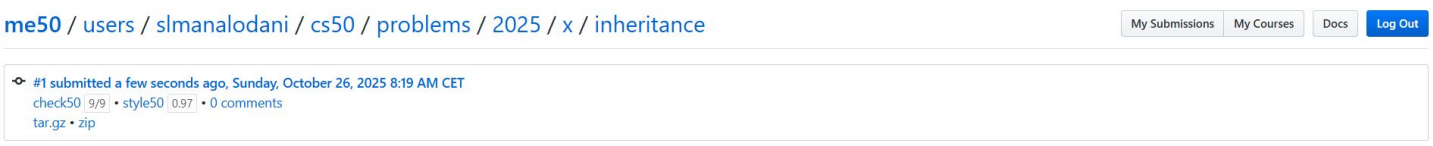
    // Print indentation
    for (int i = 0; i < generation * INDENT_LENGTH; i++)
    {
        printf(" ");
    }

    // Print person
    if (generation == 0)
    {
        printf("Child (Generation %i): blood type %c%c\n", generation, p->alleles[0],
               p->alleles[1]);
    }
    else if (generation == 1)
    {
        printf("Parent (Generation %i): blood type %c%c\n", generation, p->alleles[0],
               p->alleles[1]);
    }
    else
    {
        for (int i = 0; i < generation - 2; i++)
        {
```

```
            printf("Great-");
        }
        printf("Grandparent (Generation %i): blood type %c%c\n", generation, p->alleles[0],
               p->alleles[1]);
    }

    // Print parents of current generation
    print_family(p->parents[0], generation + 1);
    print_family(p->parents[1], generation + 1);
}

// Randomly chooses a blood type allele.
char random_allele()
{
    int r = random() % 3;
    if (r == 0)
    {
        return 'A';
    }
    else if (r == 1)
    {
        return 'B';
    }
    else
    {
        return 'O';
    }
}
```



https://submit.cs50.io/check50/53e8e6056ffcc0c2a0a2e113063af20ec00a7beb

Assignment3: speller

In the **CS50 speller assignment**, I'm supposed to implement a spell-checker by writing code in a file called `dictionary.c`. I need to complete functions that will **load** a dictionary into memory, **check** if words are spelled correctly, **return the size** of the dictionary, and **unload** it from memory. My code will be used by `speller.c` to check spelling in a text file using the dictionary I loaded.

The code:

```c
// Implements a dictionary's functionality

#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "dictionary.h"

unsigned int word_count = 0; // Tracks number of words loaded

// Represents a node in a hash table
typedef struct node
{
    char word[LENGTH + 1]; // LENGTH longest word has 45 letter + null terminator
    struct node *next;
} node;

// Choose number of buckets in hash table
const unsigned int N = 10000; /*
Collision control: With ~140k words, 10k buckets gives ~14 words per bucket on average — fast lookup.
Memory efficiency: 10k pointers around 80 KB */

// Hash table
node *table[N];

// Returns true if word is in dictionary, else false
bool check(const char *word)
{
    // Create a lowercase copy of the input word
    // This ensures case-insensitive comparison
```

```
    char lower[LENGTH + 1]; // +1 for null terminator
    int i = 0;

    // Convert each character to lowercase
    while (word[i] && i < LENGTH)
    {
        lower[i] = tolower(word[i]); // tolower handles uppercase letters
        i++;
    }
    lower[i] = '\0'; // Null-terminate the string

    // Hash the lowercase word to get its bucket index
    unsigned int index = hash(lower);

    // Start at the head of the linked list in that bucket
    node *cursor = table[index];

    // Traverse the linked list one node at a time
    while (cursor != NULL)
    {
        // Compare the current node's word with the input word
        // strcmp returns 0 if the strings are equal
        if (strcmp(cursor->word, lower) == 0)
        {
            return true; // Word found in dictionary
        }

        // Move to the next node in the list
        cursor = cursor->next;
    }

    // If we reach the end of the list without a match, word is not in dictionary
    return false;
}

// Hashes word to a number between 0 and N - 1
unsigned int hash(const char *word)
{
    unsigned int hash = 0;

    // Loop through each character in the word
    for (int i = 0; word[i] != '\0'; i++)
    {
```

```
        // Convert to lowercase for consistency
        char c = tolower(word[i]);

        // Add character value to hash
        hash += c;
    }

    // Fit hash into table size
    return hash % N; // Modulo N → keeps result inside 0–9999 bucket range.
}

// Loads dictionary into memory, returning true if successful, else false
bool load(const char *dictionary)
{
    // Open the dictionary file for reading
    FILE *file = fopen(dictionary, "r");
    if (file == NULL)
    {
        return false; // File couldn't be opened
    }

    // Buffer to store each word from the file
    char word[LENGTH + 1];

    // Read words one at a time until end of file
    while (fscanf(file, "%s", word) != EOF)
    {
        // Allocate memory for a new node
        node *new_node = malloc(sizeof(node));
        if (new_node == NULL)
        {
            fclose(file);
            return false; // Memory allocation failed
        }

        // Copy the word into the node
        strcpy(new_node->word, word);

        // Hash the word to find the correct bucket
        unsigned int index = hash(word);

        // Insert node at the beginning of the linked list
        new_node->next = table[index];
```

```
        table[index] = new_node;

        word_count++; // Count each word added to the dictionary

    }

    // Close the dictionary file
    fclose(file);

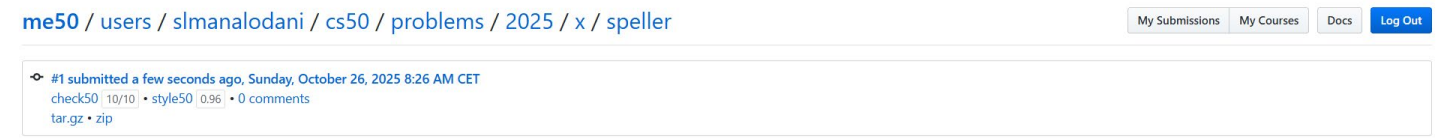
    return true; // Dictionary loaded successfully
}

// Returns number of words in dictionary if loaded, else 0 if not yet loaded
unsigned int size(void)
{
    return word_count;
}

// Unloads dictionary from memory, returning true if successful, else false
bool unload(void)
{
    // Loop through each bucket in the hash table
    for (int i = 0; i < N; i++)
    {
        node *cursor = table[i]; // Start at the head of the list

        // Traverse the linked list and free each node
        while (cursor != NULL)
        {
            node *temp = cursor;     // Save current node
            cursor = cursor->next;   // Move to next node
            free(temp);              // Free the saved node
        }
    }

    return true; // All memory successfully freed
}
```



https://submit.cs50.io/check50/d0fcbbb07d4c13f7f17d8707b2b2baa8e0e5d8c6

Embedded assignments:

CuteBot Race game:

The assignment is to program the cutebot to do three main tasks:

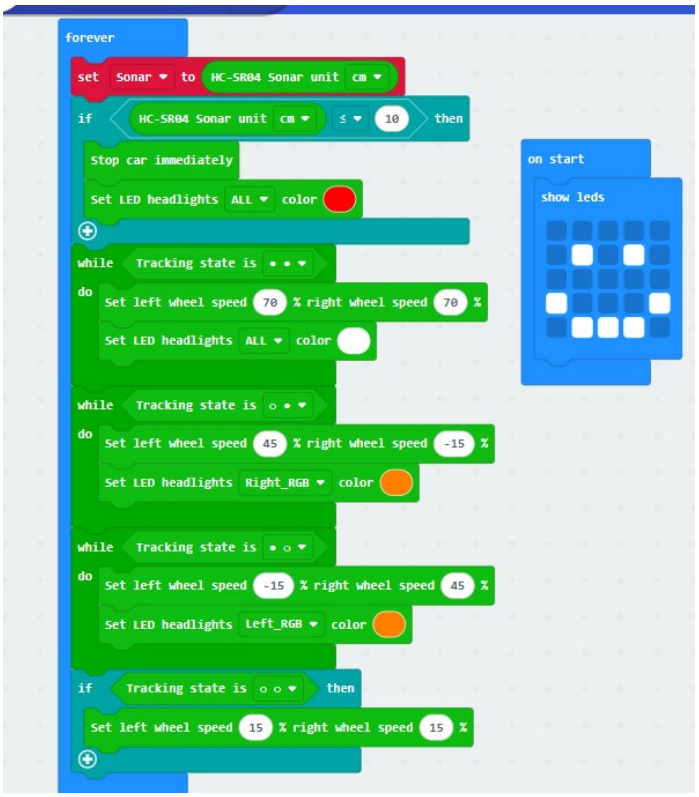
1. Follow the line using its onboard infrared sensors.
2. Give a signal with the RGB lights when it turns left or right.
3. It should stop when it detects something Infront of it by the ultrasonic sensor.

The assignment started on Tuesday 2nd of September, and I was working as a duo with my classmate Radu.

We started with programming the microbit board using this website:

(https://makecode.microbit.org/#editor)

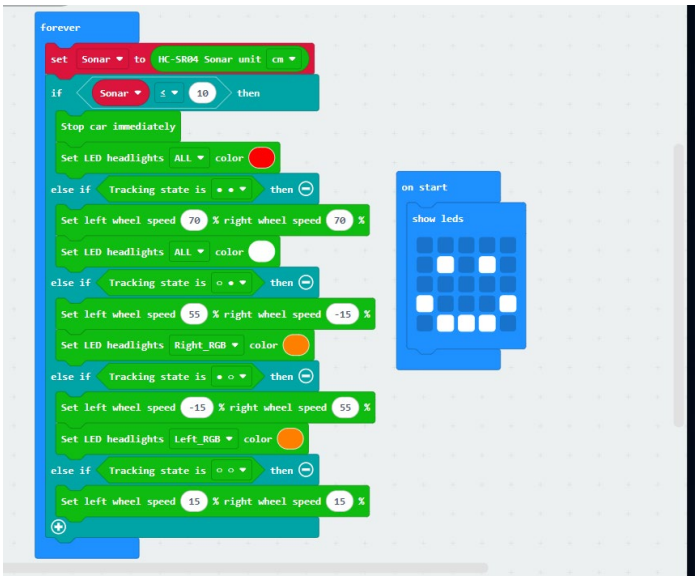
Down here is a screenshot of one of our initial blocks combinations:



We had a couple of issues there:

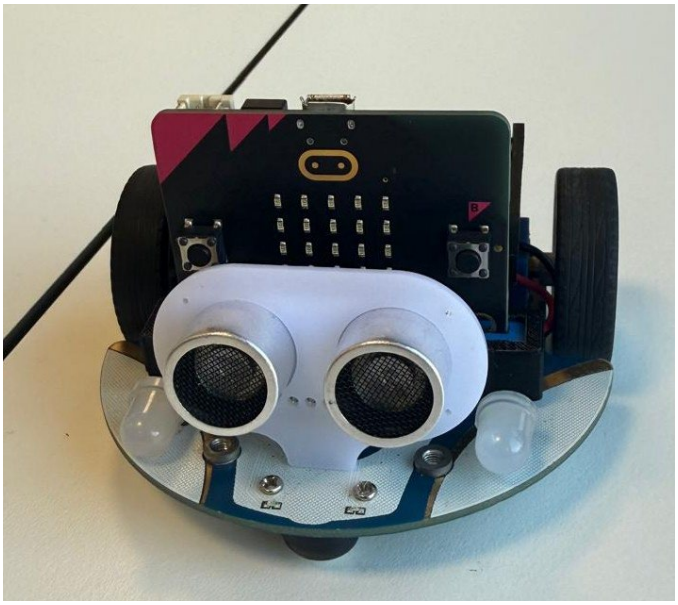
1. The ultrasonic sensor wasn't working completely.
2. Cutebot was turning left and right rapidly when it took a turn.

So, we tried a new approach to these problems. Down here is a screenshot of the latest blocks combinations.



As you can see there, we changed the (while condition do loops) into if else conditions and we corrected the sonar variable. Also, we tweaked the speed of the wheels. The result was that the cutebot now can stop when it detects something Infront of it and its more stable in taking left and right turns.

Here is a picture of our cutebot:



Mario uno:

This assignment is to use the Adruino IDE to program a Mario right-aligned pyramid and show it in the Serial monitor.

The goal is to translate cs50 assignment Mario-less code in C to another language here which is C++.

Tools:

- C++ Style Guide for Arduino projects.
  https://www.makerguides.com/c-style-guide-for-arduino-projects/
- Language reference.
  https://docs.arduino.cc/language-reference/
- Mastering the Arduino Uno R4 book.
  Chapter 2.3.1:
  Example 1: Sum of integer numbers.

The code:

```
//
//                    Mario uno
//
// This program display mario-less (creating a right-aligned pyramid)
//
//
// Author: Slman
// file   : Mario-uno
// Date   : Sep, 2025
//-------------------------------------------------------
void printRow(int spaces, int bricks) // custom function  print a row consist of spaces and
bricks
{
    for ( int i = 0; i < spaces; i++)
    {
      Serial.print(" ");                  // print spaces
    }
    for ( int i = 0; i < bricks; i++)
    {
      Serial.print("#");                  // print bricks
    }
    Serial.println();                     // print new line
 }
void setup()
 {
```

```
  Serial.begin(9600);                    // start serial communication.
  delay(1000);
 }
void loop()
{
  int h;                                 // to define h as integer
  do                                     // to keep asking for the intended answer
  {
    Serial.print("Height: ");            // display Height

     while (Serial.available() <= 0) {   // Wait for input
    }
     h = Serial.parseInt();              // Reads input as an integer and stores it in h.
    Serial.print(h);                    // prints the value back so the user can see what
they entered
    Serial.println();                   // print new line
   }
  while ( h < 1 || h > 8 );              // always positive num and not more than 8

  for ( int i = 0; i < h; i++)          // print pyramid
  {
    int nspaces = h - 1 - i;             // formula to calculate number of spaces to print
before the bricks
    int nbricks = i + 1;                 // formula to calculate number of bricks
    printRow( nspaces, nbricks);
  }
}
```

Problems that I had:

1. I didn't know where I should declare my custom function. I found the answer from this forum: https://forum.arduino.cc/t/functions-ahead-of-setup-and-loop/539324
2. For some reason if I don't add the delay, the serial monitor didn't show (Height:)

Morse code :

The assignment is to make a morse code generator using the Arduino.

- The program should be able to covert any text, numbers and special characters to morse code , and present that with LED or buzzer.

Code:

```
//
//                    Morse code generator
//
// This program generate morse code using a buzzer and external LED
//
//
// Author : Slman
// file   : Morse code generator
// Date   : 16 sep 2025
//--------------------------------------------------------------------------
// Define ON and OFF
#define ON HIGH                         // HIGH means 5V
#define OFF LOW                         // LOW means 0V

// set up the pins and Timing
const int buzzerPin = 7;                 // to tell the Arduino which pin control the buzzer
(D7)
const int ledPin = 9;                    // to tell the Arduino which pin control the
external LED (D9)
const int dotTime = 100;                 // sets the base time unit for Morse code in
miliseconds.

// Define character and Morse code arrays
char letters[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789?!.,;:+-/=@";      // the compiler turn
it into {'A','B',...,'\0'};
String morse[] = {
  ".-", "-...", "-.-.", "-..", ".", "..-.", "--.", "....", "..",
  ".---", "-.-", ".-..", "--", "-.", "---", ".--.", "--.-", ".-.",
  "...", "-", "..-", "...-", ".--", "-..-", "-.--" , "--..", "-----",
  ".----", "..---", "...--", "....-", ".....", "-....", "--...",
  "---..", "----.", "..--..", "-.-.--", ".-.-.-", "--..--", "-.-.-."
  "---...", ".--.-.", "-....-", "-..-.", "-...-", ".--.-."
```

```
};

void setup()
{
  pinMode(buzzerPin, OUTPUT);             // configure buzzerPin as an output
  pinMode(ledPin, OUTPUT);                // configure ledPin as an output
  Serial.begin(9600);                     // starts the serial monitor
  delay(2000);
  Serial.println("Type something: ");
}

void loop()
{
  // to check if the user typed something into Serial monitor
  if (Serial.available())
  {
    String msg = Serial.readStringUntil('\n');  /* read the full line of text the user typed,
up to  ((arduino uses S capital for string))
   the Enter key ('\n') and stored it in  msg variable */

    msg.toUpperCase();                    // convert to uppercase

    Serial.println(msg);

    for (int i = 0; i < msg.length(); i++)   // loop goes through each character in the msg
string
    {
      char c = msg.charAt(i);             // get the character at position i from the msg
string and stored it at c variable

      if (c == ' ')                       // Check if the character is a space
      {
        delay(dotTime * 7);              // Wait for a longer time to separate words (7 units
in Morse code)
        continue;                       // Skip the rest of the loop and move to the next
character
      }
      //search for character from the letters array
      for (int j = 0; j < strlen(letters); j++)      //gives the actual number of characters,
excluding the null terminator.
      {
        if (c == letters[j])
        {
          playMorse(morse[j]);          // play morse code for that character
```

```
          delay(dotTime * 3);           // space between letters
          break;                        // Stop searching once the matching character is
found
        }
      }
    }
  }
}

// Morse code function
void playMorse(String code)
{
  for (int i = 0; i < code.length(); i++)          // loop goes through each character in
morse code string
  {
    // turn on buzzer and LED
    digitalWrite(buzzerPin, ON);
    tone(buzzerPin, 300);             // control the tone
    digitalWrite(ledPin, ON);

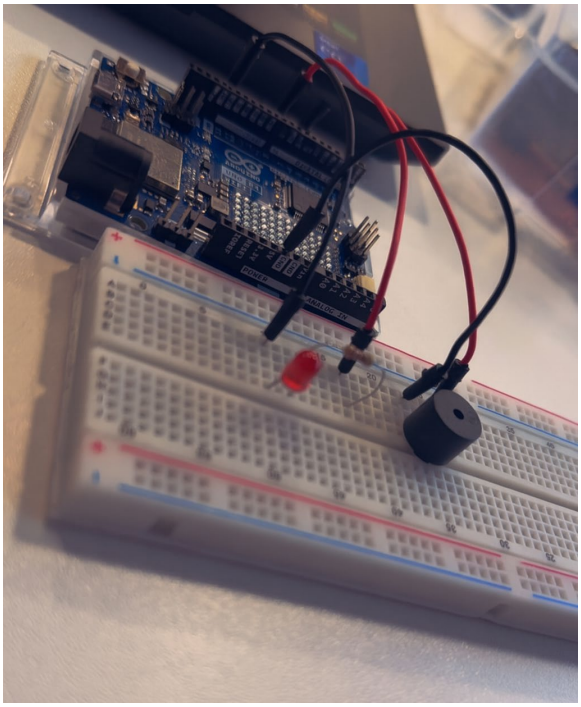
    //Hold signal for dot or dash duration
    if (code.charAt(i) == '.')
    {
      delay(dotTime);                    // Dot = short signal

    } else if (code.charAt(i) == '-')
    {
      delay(dotTime * 3);                // Dash = long signal 3 units
    }

    // turn off buzzer and LED
    digitalWrite(buzzerPin, OFF);
    noTone(buzzerPin);            // stop tone
    digitalWrite(ledPin, OFF);

    // pause between symbols
    delay(dotTime);
  }
}
```

Picture of the project:



Advance traffic lights:

Design and implement a traffic control system using an Arduino Uno that manages:

- **Two intersecting roads**
- **Two pedestrian crosswalks**
- **Two push buttons** (one for each crosswalk)
- **Dedicated traffic lights** for both vehicles and pedestrians

The code:

```
//                        Traffic Lights
//
//
//
// This program shows the traffic lights with the helps of the shift register.
//
//
//
//
// Author: Slman
// File: trafficlights
// Date: 26/ 09/ 2025
//--------------------------------------------------------------------------------
// Shift register pins
const int DATAPIN = 4;
const int LATCHPIN = 5;
const int CLOCKPIN = 6; // This pin is pulsed HIGH then LOW to tell the shift register: "Read
the bit on the data pin now."

// Pedestrian button pins
const int NORTHBUTTONPIN = 2;
const int EASTBUTTONPIN = 3;

// Flags for pedestrian requests
bool northRequest = false; // button is not pressed
bool eastRequest = false;

// EAST road light patterns
const byte EASTSEQUENCE[] = {
  B10001001, // East Red + North Red + Ped Red
  B10001011, // East Red + Yellow + North Red + Ped Red
```

```
  B10001100, // East Green + North Red + Ped Red
  B10001010, // East Yellow + North Red + Ped Red
  B10001001  // East Red + North Red + Ped Red
};

const int EASTDELAYS[] = {
  2000, 2000, 7000, 2000, 3000
};

//North road light patterns
const byte NORTHSEQUENCE[] = {
  B10011001, // North Red + Yellow + East Red + Ped Red
  B10100001, // North Green + East Red + Ped Red
  B10010001, // North Yellow + East Red + Ped Red
  B10001001  // North Red + East Red + Ped Red
};

const int NORTHDELAYS[] = {
  2000, 7000, 2000, 3000
};

// pedestrain light patterns
const byte PEDESTRAINSEQUENCE[] = {
  B01001001, // Ped Green + East Red + North Red
  B10001001  // Ped Red + East Red + North Red
};

const int PEDESTRAINDELAYS [] = {
  6000, 1000
};

void setup() {

  // Set shift register pins as outputs
  pinMode(DATAPIN, OUTPUT);
  pinMode(LATCHPIN, OUTPUT);
  pinMode(CLOCKPIN, OUTPUT);

  // Set pedestrian button pins
  pinMode(NORTHBUTTONPIN, INPUT_PULLUP);
  pinMode(EASTBUTTONPIN, INPUT_PULLUP);

  // Attach interrupts to detect button presses (falling = button press)
```

```
  attachInterrupt(digitalPinToInterrupt(NORTHBUTTONPIN), northPress, FALLING);
  attachInterrupt(digitalPinToInterrupt(EASTBUTTONPIN), eastPress, FALLING);
}

void loop() {

  // EAST ROAD SEQUENCE
   for (int i = 0; i < sizeof(EASTSEQUENCE) / sizeof(EASTSEQUENCE[0]); i++) {
    // sizeof(EASTSEQUENCE) gives total bytes of array
    // sizeof(EASTSEQUENCE[0]) gives size of one element (1 byte)
    // So this calculates number of elements in the array
    showLights(EASTSEQUENCE[i]);
    delay(EASTDELAYS[i]);
  }

  // NORTH ROAD SEQUENCE
  for (int i = 0; i < sizeof(NORTHSEQUENCE) / sizeof(NORTHSEQUENCE[0]); i++) {
    showLights(NORTHSEQUENCE[i]);
    delay(NORTHDELAYS[i]);
  }

  // Check for pedestrian request
  if (northRequest || eastRequest) {
    runPedestrianSequence();
    eastRequest = false;
    northRequest = false;
  }
}

// When North button pressed
void northPress() {
  northRequest = true;
}

// When East button pressed
void eastPress() {
  eastRequest = true;
}

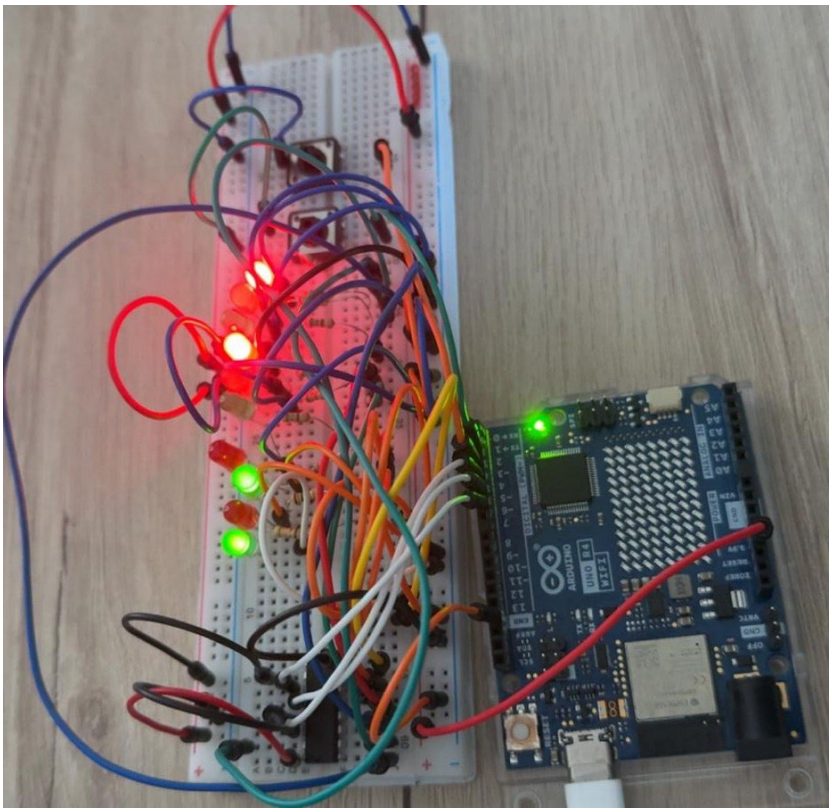
// Pedestrian crossing sequence
void runPedestrianSequence() {
  for (int i = 0; i < sizeof(PEDESTRAINSEQUENCE) / sizeof(PEDESTRAINSEQUENCE[0]); i++) {
    showLights(PEDESTRAINSEQUENCE[i]);
    delay(PEDESTRAINDELAYS[i]);
```

```
  }
}

// Send light pattern to shift registe

void showLights(byte lights) {
  digitalWrite(LATCHPIN, LOW); // Prepare to send new data
  shiftOut(DATAPIN, CLOCKPIN, MSBFIRST, lights); // Send 8 bits to shift register
  digitalWrite(LATCHPIN, HIGH); // Update outputs with new data
}
```

Picture of the project:



Displaying Uptime:

For this assignment, I'm building a **minute:second counter** using a **4-digit 7-segment display**. The goal is to show how much time has passed since the system started, in the format **MM:SS**.

The code:

```
//                  Display Uptime
//
// A 4-digit 7-segment display shows how many minutes and seconds have passed since startup.
// Uses timer interrupts and multiplexing to refresh digits smoothly.
// Decimal points between digit 2 and 3 simulate a colon separator blinking every second.
//
// author: SLMAN
// file: Display_Uptime
// date: 6/10/2025
//=========================================

#include "FspTimer.h"           // Include the timer library

FspTimer MyTimer;              // Create a timer object - PascalCase(UpperCamelCase)

// Define constants for readability
#define ON HIGH
#define OFF LOW
#define Enable HIGH
#define Disable LOW

// Define LED pins for 7-segment display segments (g to a)

int LED[] = {8, 7, 6, 5, 4, 3, 2};
/* LED[]: Array holding pin numbers for each segment.
Segments are ordered from g to a.*/

const int DP_PIN = A0; // Separate pin for controlling the shared DP segment

// Updated SEG array using binary format (7 bits for a–g)
const byte SEG[] = {
  B00111111, // 0 → a b c d e f
  B00000110, // 1 → b c
  B01011011, // 2 → a b d e g
  B01001111, // 3 → a b c d g
```

```
  B01100110, // 4 → b c f g
  B01101101, // 5 → a c d f g
  B01111101, // 6 → a c d e f g
  B00000111, // 7 → a b c
  B01111111, // 8 → all segments
  B01101111  // 9 → a b c d f g
};

// Pins for digit control (DIG1 to DIG4)
unsigned char DIGITS[] = {9, 10, 11, 12};
/*unsigned: Type modifier means the variable can only hold non-negative values.
char with unsigned when combined, char stores a small integer type (0 to 255).
Here, used to save memory since pin numbers are small integers.*/

// Variables to hold each digit of the counter
unsigned int MSD, MID2, MID1, LSD;
/* MSD: Most Significant Digit (leftmost digit) → tens place of minutes.
   MID2: Middle Digit 2 → units place of minutes.
   MID1: Middle Digit 1 → tens place of seconds.
   LSD: Least Significant Digit → units place of seconds. */

unsigned char Pattern; // Hold the binary pattern for a digit on the 7-segment display.
int totalSeconds = 0;  // Counter value in seconds (0–5999 → 99:59)
int flag = 0;          // Used to cycle through digits - tells us which digit we're currently
working on.

uint8_t TimerType;     /* uint8_t: Stands for Unsigned Integer, 8 bits.
Its a Data type that can store numbers from 0 to 255. Compared to int (which is usually 32
bits)*/
// TimerType: will hold a code or label that tells the system which kind of timer to use.

int8_t TimerIndex; // int8_t: Stands for Signed Integer, 8 bits - Store values from -128 to
+127.
//TimerIndex: Used to select and configure the correct timer from the available hardware
timers.

unsigned long lastTime = 0; // last update time -  unsigned long range (0 to ~4.29B) this is
good for time tracking

// Setup runs once at startup
void setup() {

  for (int i = 0; i < 7; i++)
```

```
  {
    pinMode(LED[i], OUTPUT);  // Segment pins a-g
    pinMode(DP_PIN, OUTPUT);  // Separate DP pin
  }

  for (int i = 0; i < 4; i++)
  {
    pinMode(DIGITS[i], OUTPUT);                    // Digit control pins
    digitalWrite(DIGITS[i], Disable);              // Turn all digits off
  }

  StartTimer(200);                                 // Start timer at 200 Hz (200 cycles
per second)
}

// Main loop runs repeatedly
void loop() {
  if (millis() - lastTime >= 1000) { // 1 sec passed?
    lastTime = millis();            // save current time
    totalSeconds++;                 // add 1 second
    if (totalSeconds >= 6000)      // reset after 99:59
      totalSeconds = 0;
  }
}

// Display a binary pattern on the 7-segment LEDs
void Display(int No, int L) { /*
int No → the binary pattern to display (like B00111111 for digit 0).
int L → the number of segments to update (7 for a-g).*/

  int i, m, j; /*
  i → loop counter for segment positions.
  m → used to calculate bit position.
  j → used to isolate individual bits from No.*/

  m = L - 1; /*
  So we can start checking bits from the leftmost segment (bit 6) and move right.
  It helps match each bit in the pattern to the correct LED segment.*/

  for (i = 0; i < L; i++) { // loop runs once for each LED segment.

    j = 1;
    for (int k = 0; k < m; k++) j = j * 2;
```

```
    if ((No & j) != 0) // This checks if bit m in No is ON
      digitalWrite(LED[i], ON);
      else
      digitalWrite(LED[i], OFF);

    m--; // We move to the next bit to the right for the next loop.
  }
}

// Timer interrupt service routine (called every 5ms)
void TimerCallback(timer_callback_args_t __attribute((unused)) *p_args) { /*
timer_callback_args_t → This is a data type that holds info about the timer event (like which
timer fired).
*p_args → This is a pointer to that data.
__attribute((unused)) → This tells the compiler: “Yes, I know I’m not using p_args, and
that’s okay — don’t give me a warning.”*/

  //Break the counter into MM:SS format
  int minutes = totalSeconds / 60; // ex: if totalseconds is 1 then 1/60 = 0 integer division
  int seconds = totalSeconds % 60; // if totalseconds is 1 then remainder of 1 divided by 60
is just 1

  MSD  = minutes / 10;     // Tens place of minutes
  MID2 = minutes % 10;     // Units place of minutes
  MID1 = seconds / 10;     // Tens place of seconds
  LSD  = seconds % 10;     // Units place of seconds

  int digits[] = {MSD, MID2, MID1, LSD};

  bool blinkState = (totalSeconds % 2 == 0); /*
  blinkState: toggles every second.
  If totalSeconds is even → blinkState = true → dots ON.
  If odd → blinkState = false → dots OFF.
  This creates a blinking colon effect between digits 2 and 3. */

  //Here is the multiplexing logic, which handles lighting up one digit at a time.
  digitalWrite(DIGITS[(flag + 3) % 4], Disable); /*
   Turn off previous digit that was previously active
   % 4 (modulo 4) locks flag into the range 0–3.
   Math: if flag 0: 0 + 3 = (3 % 4) → step 1) 3/4 = Quotient = 0 : 4 * 0 = 0. step 2) 3 - 0 =
3 and that's the remainder */
```

```
  Pattern = SEG[digits[flag]]; /*
   Step 1: digits[flag] gives the number to show (like 3);
   Step 2: SEG[...] gives the LED pattern for that number; result is stored in Pattern */

  Display(Pattern, 7);                          // Show digit with 7 segments (a-g only)

  // Blink DP between digit 2 and 3 every second (colon effect)
  if (flag == 1 && blinkState) // DP ON when refreshing digit 2 and blinkState is true
    digitalWrite(DP_PIN, ON);  // Turn ON DP
  else
    digitalWrite(DP_PIN, OFF); // Turn OFF DP

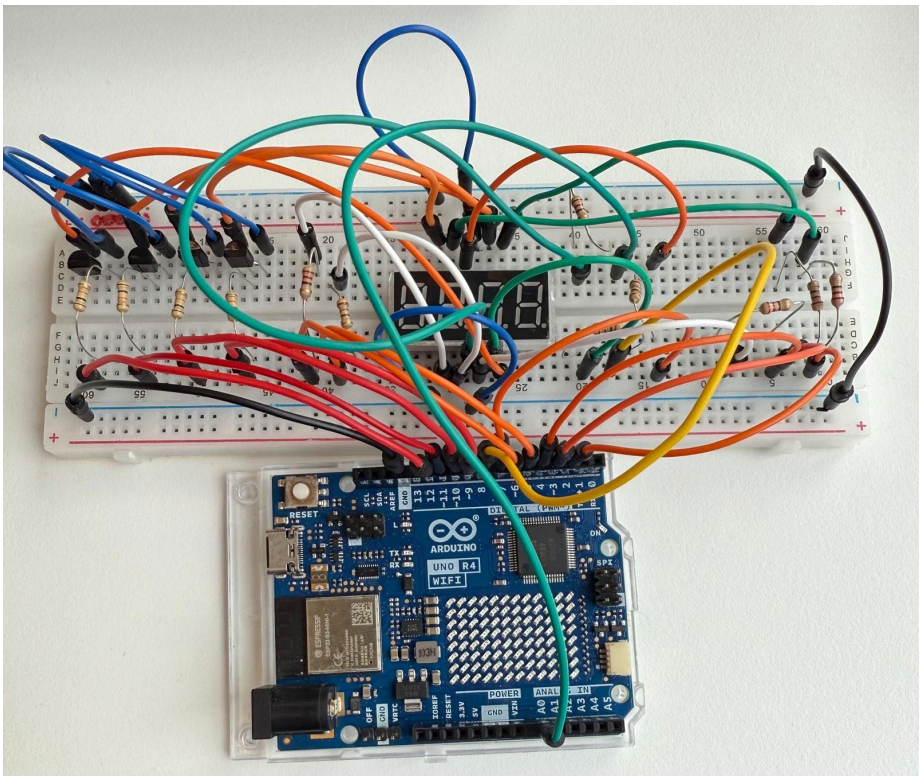
  digitalWrite(DIGITS[flag], Enable);           // Turn on current digit
  flag = (flag + 1) % 4; /*
  Cycle flag from 0 to 3.
  Move to the next digit (0 → 1 → 2 → 3 → 0 → ...)
  the looping behavior comes from the timer interrupt, which calls TimerCallback() every 5
milliseconds.*/
}

// Start and configure the timer at a given frequency
void StartTimer(float freq) {
  TimerType = GPT_TIMER; // Use a General Purpose Timer (GPT) for regular interrupts
  TimerIndex = FspTimer::get_available_timer(TimerType); // Get a free timer index from the
hardware

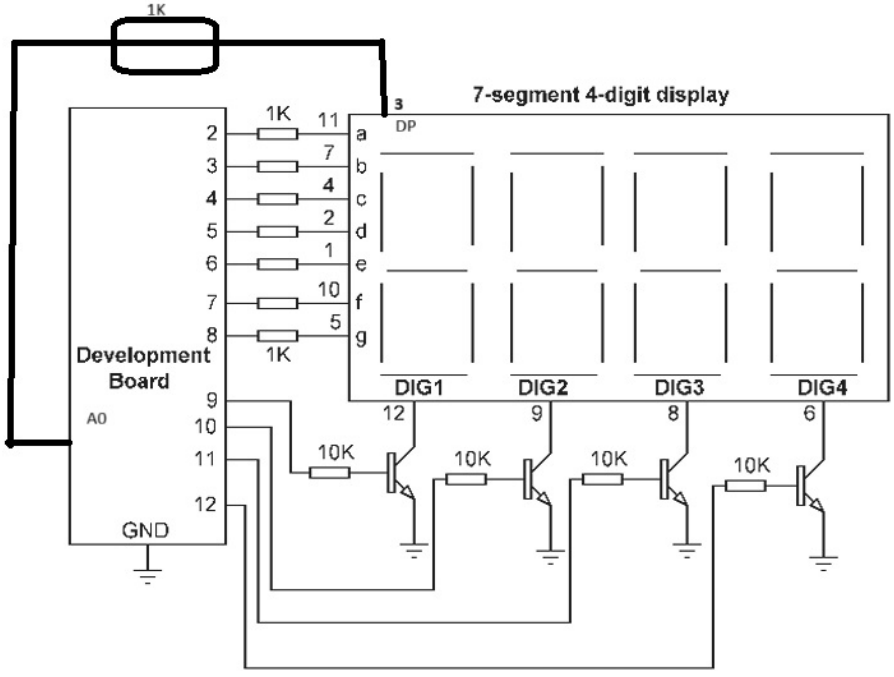
  if (TimerIndex == 0) {
    FspTimer::force_use_of_pwm_reserved_timer(); // Force use of a reserved timer if none are
free
    TimerIndex = FspTimer::get_available_timer(TimerType); // Try again to get a valid timer
index
  }

  // Set up the timer to run in periodic mode — it will call TimerCallback() repeatedly at
the given frequency
  MyTimer.begin(TIMER_MODE_PERIODIC, TimerType, TimerIndex, freq, 0.0f, TimerCallback);
  MyTimer.setup_overflow_irq(); // Sets up the timer so it runs TimerCallback() automatically
after each counting cycle
  MyTimer.open(); // Applies the timer settings and prepares it for use — but doesn't start
counting yet
  MyTimer.start(); // Starts the timer — it now begins counting and will repeatedly call
TimerCallback() at the set frequency
}
```

Picture of the project:



Schematic:



(Realtime) Jumbo Clock:

For this assignment, I'm building a **jumbo digital clock** that displays the current time in **HH:MM format**, with **seconds** shown separately. The output will appear on a **16x2 LCD screen** connected to the Arduino via an **I2C controller board** soldered to the back of the LCD.

The code:

```
//                          Jumbo clock
//
//         This program display a jumbo clock in HH:MM format with
//         seconds in a LCD display connected to the arduino by an I2C
//         controller board soldered to the back of the LCD.
//
//
// Auther: Slman
// File: jumbo_clock
// Date: 16/10/2025
//---------------------------------------------------------------

#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2); // Initialize LCD with I2C address 0x27 and size 16x2

// ---------------------- Custom Characters ----------------------
// Each byte array defines a 5x8 pixel custom character for LCD segments
byte line_vertical_right[8] = { B00001, B00001, B00001, B00001, B00001, B00001, B00001,
B00001 };
byte line_vertical_left[8]  = { B10000, B10000, B10000, B10000, B10000, B10000, B10000,
B10000 };
byte c_shape[8]             = { B11111, B10000, B10000, B10000, B10000, B10000, B10000,
B11111 };
byte c_shape_reversed[8]    = { B11111, B00001, B00001, B00001, B00001, B00001, B00001,
B11111 };
byte frame_closed[8]        = { B11111, B10001, B10001, B10001, B10001, B10001, B10001,
B11111 };
byte frame_open_top[8]      = { B10001, B10001, B10001, B10001, B10001, B10001, B10001,
B11111 };
byte frame_open_bottom[8]   = { B11111, B10001, B10001, B10001, B10001, B10001, B10001,
B10001 };
byte sevenShape[8]          = { B11111, B00001, B00001, B00001, B00001, B00001, B00001,
B00001 };

// ---------------------- Digit Definitions ----------------------
```

```
// Each digit is defined by two custom characters: (top row, bottom row)
const byte DIGIT_0[2] = {6, 5};
const byte DIGIT_1[2] = {0, 0};
const byte DIGIT_2[2] = {3, 2};
const byte DIGIT_3[2] = {3, 3};
const byte DIGIT_4[2] = {5, 0};
const byte DIGIT_5[2] = {2, 3};
const byte DIGIT_6[2] = {1, 4};
const byte DIGIT_7[2] = {7, 0};
const byte DIGIT_8[2] = {4, 4};
const byte DIGIT_9[2] = {4, 0};

// Lookup table for digits 0-9 - here we have an array of pointers and each pointer point
// to a byte[2] that define digit's top and bottom.
const byte *digits[] = {
  DIGIT_0, DIGIT_1, DIGIT_2, DIGIT_3, DIGIT_4,
  DIGIT_5, DIGIT_6, DIGIT_7, DIGIT_8, DIGIT_9
};

// ---------------------- Time Variables ----------------------
int hour = 0;
int minute = 0;
int seconds = 0;
unsigned long lastSecondTick = 0;   // Tracks last second update - unsinged long used her for
timing especially with millis()
unsigned long lastColonToggle = 0;  // Tracks last colon blink
bool colonVisible = true;           // Colon visibility state

// ---------------------- Setup ----------------------
void setup() {
  lcd.init();         // Initialize LCD
  lcd.backlight();    // Turn on backlight

  // Load all custom characters into LCD memory (slots 0-7)
  lcd.createChar(0, line_vertical_right);
  lcd.createChar(1, line_vertical_left);
  lcd.createChar(2, c_shape);
  lcd.createChar(3, c_shape_reversed);
  lcd.createChar(4, frame_closed);
  lcd.createChar(5, frame_open_top);
  lcd.createChar(6, frame_open_bottom);
  lcd.createChar(7, sevenShape);

  // Generate random starting time
```

```
  int pin = random(0, 3); // pick 0, 1, or 2
  int seed = analogRead(A0 + pin); // reads A0, A1, or A2
  randomSeed(seed);
  // Use analog noise to seed randomness
  hour = random(0, 24); // (min, max)
  minute = random(0, 60);
  seconds = random(0, 60);

  // Initial display setup
  updateTimeDisplay();        // Show HH:MM digits
  showColon(true, 7);       // Show colon at center
  lcd.setCursor(13, 1); lcd.write('0'); // Show seconds tens digit
  lcd.setCursor(14, 1); lcd.write('0'); // Show seconds units digit
}

// ---------------------- Main Loop ----------------------
void loop() {
  unsigned long currentMillis = millis(); // Get current time in ms

  // Update seconds every 1000 ms
  if (currentMillis - lastSecondTick >= 1000) { // if 1 sec passed
    lastSecondTick = currentMillis;
    tickClock(); // Advance time and update display
  }

  // Blink colon every 1000 ms
  if (currentMillis - lastColonToggle >= 1000) {
    lastColonToggle = currentMillis;
    colonVisible = !colonVisible;
    // Toggle colon state - Flips the value of colonVisible.
    //If it was true, it becomes false. If it was false, it becomes true.
    showColon(colonVisible, 7);       // Update colon display
  }
}

// ---------------------- Custom Functions ----------------------

// Show a digit using two custom characters at a given column
void showDigit(const byte digit[2], int col) {
  for (int i = 0; i < 2; i++) {
    lcd.setCursor(col, i);     // Set cursor to top or bottom row
    lcd.write(digit[i]);       // Write custom character
  }
}
```

```
// Renders the colon (center dots) based on visibility flag
void showColon(bool visible, int col) {
  char symbol = visible ? '.' : ' ';    // Use '.' or blank
  //ternary statement If visible == true, then symbol = '.'
  //If visible == false, then symbol = ' '
  lcd.setCursor(col, 0); lcd.write(symbol); // Top row dot
  lcd.setCursor(col, 1); lcd.write(symbol); // Bottom row dot
}

// Updates the HH:MM digits on the LCD
void updateTimeDisplay() {
  int h1 = hour / 10;
  int h2 = hour % 10;
  int m1 = minute / 10;
  int m2 = minute % 10;

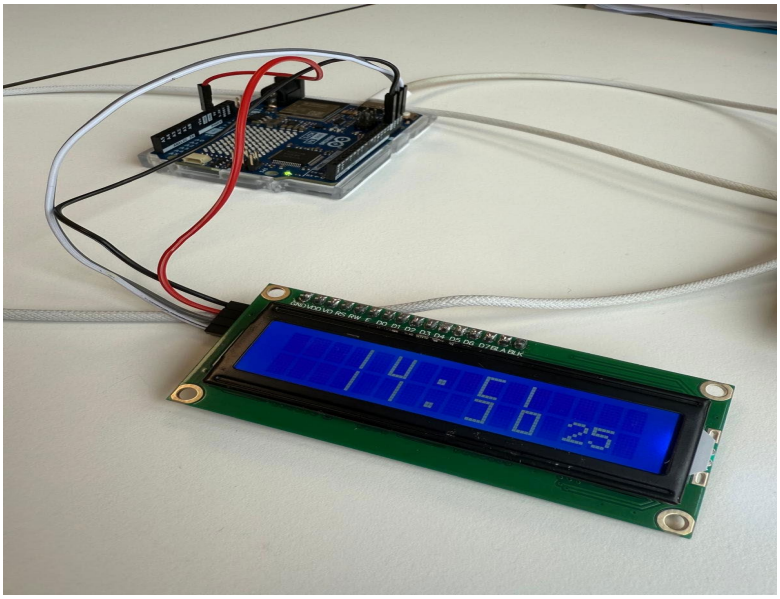
  showDigit(digits[h1], 3); // Hour tens at column 3
  showDigit(digits[h2], 5); // Hour units at column 5
  showDigit(digits[m1], 9); // Minute tens at column 9
  showDigit(digits[m2], 11); // Minute units at column 11
}

// Advances time by one second and updates seconds display
void tickClock() {
  seconds++;
  if (seconds >= 60) {
    seconds = 0;
    minute++;
    if (minute >= 60) {
      minute = 0;
      hour++;
      if (hour >= 24) hour = 0;
    }
    updateTimeDisplay(); // Refresh HH:MM digits
  }

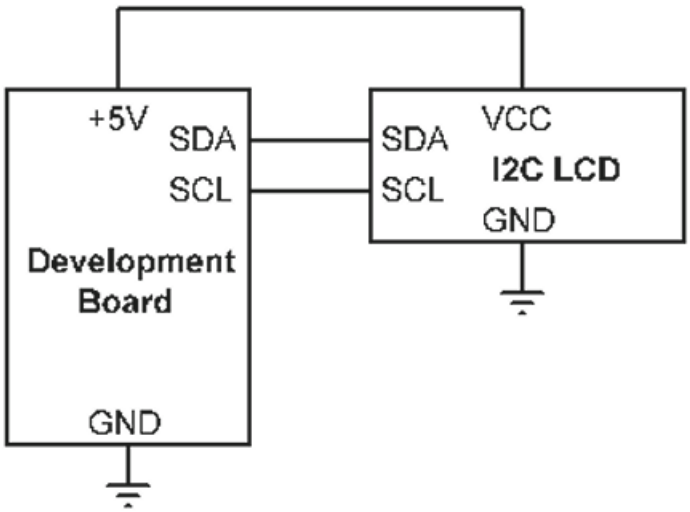
  // Update seconds digits at bottom-right corner
  int s1 = seconds / 10;
  int s2 = seconds % 10;
  lcd.setCursor(13, 1); lcd.write(s1 + '0'); // Tens digit (+ '0' is a way to convert a
number to its character form)
  //'0' is the ASCII code for the character '0' → that's 48
  lcd.setCursor(14, 1); lcd.write(s2 + '0'); // Units digit
```

```
}
```

Picture of the project:



Schematic:



Weather station:

For this assignment, I'm building a **weather station** using an Arduino and a **16x2 I2C LCD display**. The goal is to show both **indoor and outdoor temperatures** in degrees Celsius, along with **humidity**, using sensor data.

The code:

```
//                          weather_station
//
//         This program displays indoor and outdoor temperature in °C,
//         and humidity with a droplet icon on a 16x2 LCD screen.
//         The LCD is connected to the Arduino via an I2C controller.
//         Screen switches every 5 seconds between temperature, humidity,
//         and min/max indoor temperature views.
//
// Author: Slman
// File: weather_station
// Date: 26/10/2025
//-----------------------------------------------------------------
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// LCD setup
LiquidCrystal_I2C lcd(0x27, 16, 2); // I2C address

// DHT11 setup
#define DHTPIN 2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// LM35 setup
#define LM35PIN A0

// Custom droplet character (5x8)
byte droplet[8] = {
  B00100,
  B01110,
  B11111,
  B11111,
  B11111,
  B01110,
  B00100,
  B00000
```

```
};

// Timing logic
unsigned long lastSwitch = 0;
int screenIndex = 0; // 0 = temp, 1 = humidity, 2 = min/max

// Min/Max tracking
float minTemp = 100.0;
float maxTemp = -100.0;

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.createChar(0, droplet);
  dht.begin();
}

void loop() {

  float innerTemp = dht.readTemperature();
  float humidity = dht.readHumidity();
  float outerTemp = readLM35Celsius(LM35PIN);

  // Skip loop if sensor failed (NaN reading)
  // Wait 2s before retrying to avoid bad data
  if (isnan(innerTemp) || isnan(humidity)) {
    delay(2000);
    return;
  }

  // Update min/max indoor temperature
  if (innerTemp < minTemp) minTemp = innerTemp; /*
  Starts very high, so any real indoor temperature (say, 22.5°C) will be lower.
  This guarantees that the first reading becomes the new minimum.*/
  if (innerTemp > maxTemp) maxTemp = innerTemp;

  // Switch screen every 5 seconds
  if (millis() - lastSwitch >= 5000) {
    screenIndex = (screenIndex + 1) % 3;
    lcd.clear();
    lastSwitch = millis();
  }
```

```
  // Display appropriate screen
  if (screenIndex == 0) {
    showTemperature(innerTemp, outerTemp);
  } else if (screenIndex == 1) {
    showHumidity(humidity);
  } else {
    showMinMax(minTemp, maxTemp);
  }

  delay(2000);
}

//---------------------------------------------------------------
// Custom Functions
//---------------------------------------------------------------

float readLM35Celsius(int pin) {
  int raw = analogRead(pin);
  float mV = 5000.0 * raw / 1023.0;
  return mV / 10.0;
}

void showTemperature(float inTemp, float outTemp) {
  lcd.setCursor(0, 0);
  lcd.print("In:  ");
  lcd.print(inTemp, 1);
  lcd.print((char)223);
  lcd.print("C");

  lcd.setCursor(0, 1);
  lcd.print("Out: ");
  lcd.print(outTemp, 1);
  lcd.print((char)223);
  lcd.print("C");
}

void showHumidity(float humidity) {
  lcd.setCursor(0, 0);
  lcd.write(byte(0)); // Droplet icon

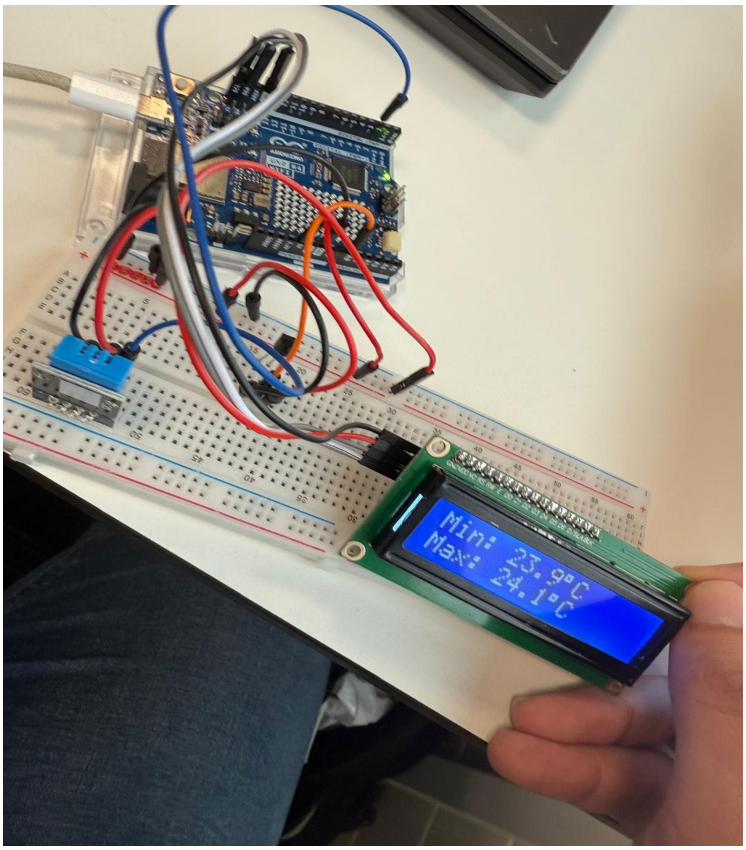
  lcd.setCursor(0, 1);
  lcd.print("Humidity: ");
  lcd.print(humidity, 1);
  lcd.print("%");
```

```
}

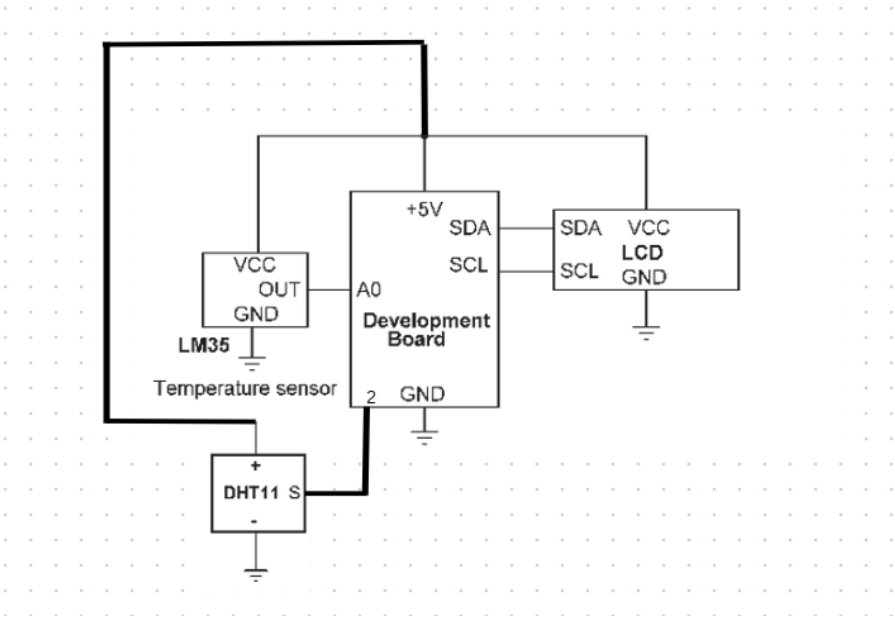
void showMinMax(float minT, float maxT) {
  lcd.setCursor(0, 0);
  lcd.print("Min: ");
  lcd.print(minT, 1);
  lcd.print((char)223);
  lcd.print("C");

  lcd.setCursor(0, 1);
  lcd.print("Max: ");
  lcd.print(maxT, 1);
  lcd.print((char)223);
  lcd.print("C");
}
```

Picture of the project:



Schematic:

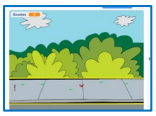

+5V
SDA
SCL
SDA
VCC
LCD
SCL
GND
VCC
OUT
GND
A0
Development
Board
LM35
Temperature sensor
2
GND
+
DHT11 S
-

Professional Skills:

Weekly Reflection Assignment:

Week 0:

This was my first week in the university and the teachers did a great job in explaining what we must do during the week. I had two projects making a game using Skratch and programming the cutebot.



For the first project, I had to do it alone and it was challenging. Thinking about what type of game I should go for was hard because I didn't want it to be simple but at the same time, I can't make it very complex, so I decided to go for something in the middle. I've watched YouTube videos to solve some problems I had with the characters animations. I've submitted the game on Thursday and if there is anything I could've done to make it better properly is by adding a starting screen.



For the second project, I had to do it with my classmate Radu. It was interesting because this was the first time that I met him. Working as a team was at first awkward because we were using his laptop on the first day and I felt like I was not helping enough, so I kept giving suggestions to the codes we used for example I thought it would be a good idea if we make the cutebot show a smile face while moving. Also, I fixed a problem we had with the ultrasonic sensor, so when I went home and reviewed our codes it appeared that we were using the wrong code for the sensor. Overall, our cutebot was doing all the tasks that the teacher asked for. On Friday we had a race with our classmates, and I was a little bit disappointed with our cutebot because it lost track of the line.

Week 1:

In that week I had 3 assignments from CS and one from Embedded Systems. The first assignment from CS was simple. I had to make a program using VS code to ask the user for his name and reply with hello, (username).

The other 2 assignments from CS were the challenging ones. Starting with Mario less where I had to make a program using VS code to ask the user for the height to draw based on that a right-aligned pyramid.

The hardest part was making the pyramid to be right-aligned and the answer for that was through introducing spaces to the code. This is picture of the pyramid when the user type 8 for the height.

```
$ cd mario-less
mario-less/ $ ./mario
Height: 8
       #
      ##
     ###
    ####
   #####
  ######
 #######
########
```

After that assignment I decided to do my Embedded system assignment because it was related to it.

It was called Mario uno. I had to use the same code from Mario-less which was in C language and translate it to C++ language that fit the Arduino to show that same right-aligned pyramid on the serial monitor. The assignments from CS so far have been clear to me because I can see what I need to do in cs50 website and follow some instructions to start my assignment.

However, with embedded systems you hear the assignment from the teacher and if you have a question, you can ask them about it. I must be always ready to make notes when it comes to embedded systems.

Week 2:

Unfortunately, I couldn't attend the Monday classes, because I went to pick up my father from the Eindhoven airport. Therefore, I missed the embedded system class which is very important to know what the objectives for that week assignment are. I talked to my classmate Nickolas, and he was very helpful.

The assignment was to make a morse code generator and present the code with buzzer or LED.

When I tried to submit the project, I got some feedback from the teacher that my generator didn't support special characters for example: (!) Or (?). That was easy to fix and added to my code.

For the CS assignments I had 3, and I've already submitted the readability and scrabble assignments which were relatively challenging, because I were using new arguments for example: strlen() this was the most common argument that I used that week for all the assignments, and what unique about it was that it works very well with arrays because it goes through each character in the string.

My last assignment was Caesar to encrypt a text, and I've finished it in the weekend. All the assignments were relatively similar in a way that you need to make use of arrays.

Week 3:

I was sick during that week, so working on my assignments took more time than it usually takes.

For the Embedded systems project, we were asked to make a traffic light. I tried to finish coding and then wire and connect everything in one go, but that didn't work so I had to reassemble it and check and connect each part. Connecting everything and trying to make it work at the first try is not efficient. From now on I should test each segment to see if there are any problems before moving forward so I can keep track and solve any issue before it overlaps with another. When I submitted it, I got feedback from Teacher Bart regarding my code. It was repetitive, so I had to implement and use arrays to make it short and enhance the readability.

For the CS the first two assignments were not that hard, while the third one was really challenging to code. I had to go through the lecture and the shorts a couple of times to be able to do it. Unfortunately, I couldn't submit any of them during that week but luckily, we still have another week to submit them.

Week 4:

We had a scavenger hunt which was a written test. I was a bit stressed about it because it was my first test in this period. I prepared for it by going through all the codes that I've written in the past weeks and that went well. It wasn't very difficult. I had two assignments for CS Filter less and Recover. Both were very interesting. In filter less we were able to use different filters for images, for example grey and blurred filters. It was very interesting to see how coding these filters works. For Recover, the goal was to recover images from a memory card, and I was able to recover 50 images.

For Embedded systems, the project was to make a Displaying Uptime in a 4-digit display. I got a lot of feedback regarding that project. For example, I made a countdown display at first and that was totally different than what we are asked to do, and I used different resistors in my project making the display lighting inconsistence and I forgot the to add a colon in the middle of the display. After receiving this feedback, I fixed the code and turned the countdown to an uptime display, and I asked the teacher for extra resistors to make the display lighting consistence. Also adding the colon wasn't very hard.

Week 5:

I had a group project with 4 classmates for CS. We made a WhatsApp group so we could communicate and assign the tasks between us. My task was to code a function that asks the user for 2 numbers and multiply them. I did get feedback from the teacher that we need to add the READ me file, so we are working on it. For the other assignment, it was called Inheritance, and we need to finish coding a program that makes a child with a random blood type inherited from his parents. It was quite complicated to code but following the hints and going back to the lecture I was able to finish coding the program.

For Embedded Systems, the project was to make a Jumbo clock in an LCD screen. It was very fun because we got to create our own fonts to represent the digits for the clock which took a very long time to make.

I got feedback from the teacher that my code looks clean, which motivates me to keep improving my coding style.

Also, He told me to add a second's counter to the bottom right of the screen and put spaces between the digits to use the whole space of the LCD. I did implement those to my Jumbo clock by altering the code, and it wasn't hard.

Week 6:

I had to make a weather station for Embedded Systems. The LCD should show the inner and outer temperatures and humidity. Wiring and coding for this project was simple following the book. I used the project to measure the humidity and temperature in my room, so it was very useful and fun to make.

I had an issue with reading the inner temperature, so the teacher helped me fix that by changing the type of sensor that I was using. That experience taught me how important it is to choose the right components and how even small hardware changes can affect the whole system.

For CS, I had one assignment called *speller*, and it was by far the hardest assignment. I did a lot of research to be able to finish it. I learned a lot about how dictionaries work behind the scenes and how to manage memory efficiently.

I still think even after completing the assignment, I need to learn more about pointers and data structures. I am planning to watch more videos on YouTube to be more familiar and comfortable with them. I also want to try building small projects that use linked lists or hash tables to reinforce my understanding.

The Yellow Card:

P1. Computer Science

Name	Student nr.	Group	Year
Slman hameed Mohamed	5638364		

The student is responsible for safe keeping of this card

Assignments

Wk	Subject	Rating	Date	Signature
1.1	Scratch	(V) O	10 SEP 2025	
1.2	Mario less **and/or** Mario more	U (G) V O	12 SEP 2025	
	Cash **and/or** Credit	U (G) V O	12 SEP 2025	
1.3	Scrabble	(V) O	18 SEP 2025	
	Readability	(V) O	18 SEP 2025	
	Ceasar **and/or** Substitution	U (G) V O	22 SEP 2025	
1.4	Sort	(V) O	03 OCT 2025	
	Plurality	(V) O	03 OCT 2025	
	Runoff **and/or** Tideman	U (G) V O	03 OCT 2025	
1.5	Scavenger hunt	(U) G V O	03 OCT 2025	10
	Filter less **and/or** Filter more	U (G) V O	10 OCT 2025	
	Recover	(V) O	10 OCT 2025	
1.6	Git Together	(V) O	16 OCT 2025	Redhe
	Inheritance	(V) O	16 OCT 2025	
1.7	Speller	(V) O	27 OCT 2025	
	Check, check, triple check	(V) O	28 OCT 2025	
1.8	**Completing Portfolio**			
T1	**Assessment**			

U = Ultimate, G = Good, V = Valid, O = Off standard

This card is proof that the above assignments are (partially) completed and should be:

- signed off during guided ateliers by the assigned teacher(s), in the same week, a week before, or a week after
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment

NHL STENDEN
university of applied sciences

P1. Embedded Systems

Name	Student nr.	Group	Year
Slman Hamood Mohamed	5638364		

The student is responsible for safe keeping of this card

Assignments

Wk	Ch	Subject	Rating	Date	Signature
1.1	1	CuteBot Race game	(V) O	5 SEP 2025	
1.2	2	Mario Uno *(example 1 to 18)*	U G (V) O	12 SEP 2025	M. Bara
1.3	3	Morse code generator *(project 1 to 13)*	U G (V) O	18 SEP 2025	M. Bara
1.4	3	(Advanced) Traffic lights *(proj. 14 to 20)*	U G (V) O	26 SEP 2025	Bara
1.5	4	Displaying Uptime *(project 1 to 6)*	(V) O	8 OCT 2025	M. Bara
1.6	5	(Realtime) Jumbo Clock *(project 1 to 6)*	U G (V) O	16 OCT 2025	M. Bara
1.7	6	Weather Station *(project 1 to 12)*	U G (V) O	27 OCT 2025	M. Bara
1.8	-	Completing Portfolio			
T1	-	**Assessment**			

U = Ultimate, G = Good, V = Valid, O = Off standard

Feedback:
1.6 nice clean code!

Validation
27 OCT 2025

This card is proof that the above assignments are (partially) completed and should be:

- signed off during guided ateliers by the assigned teacher(s), in the same week, a week before, or a week after
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment